Informatica Teorica
Appunti

Giovanni "Qua' Qua' dancer" Palma
e
Alex "Morbidelli WhatsApp" Basta

Contents

Chapter 1 **Introduzione a problemi e indecidibilita** — Page

1.1 Halting Problem
Il Metodo Diagonale — • Dimostrazione Halting Problem —

1.2 Tipi di problemi
Problemi di decisione —

1.3 Linguaggi
Definizioni — • Automi — • Macchia di Turing —

Chapter 2 **Macchine di Turing e Linguaggi** — Page

2.1 Linguaggi
Definizioni — • Automi —

2.2 Macchia di Turing
Definizioni formali — • Esercizi —

2.3
Esercizi — • Macchine Non Deterministiche —

2.4 Macchine di turing universali
Codifica della macchina di turing — • Definizione formale — • Processo di simulazione —

2.5 Problemi Ricorsivi e Ricorsivamente Enumerabili
Enumerare stringhe e macchine — • Vettore caratteristico e argomento diagonale — • Proprietà sui linguaggi —

2.6 Linguaggio Universale e Problema della Fermata
Il complemento di $\mathcal{L}_d$ — • Il Problema della Fermata — • La variante $HALT_\varepsilon$ —

Chapter 3 **Riduzioni** — Page

3.1 Teorema sulle riduzioni

3.2 Esempio: $\mathcal{L}_{ne}$

3.3 Corrispondenza di Post
Definizione — • Appartenenza a RE — • Non-appartenenza a R —

3.4 Il Problema del Tassellamento
Definizione informale — • Formalizzazione — • Indecidibilita' del Tassellamento —

3.5 Teoria della Ricorsione: gerarchia RE/R/co-RE
Riepilogo delle classi — • Teoremi fondamentali — • Il problema $\text{HALT}_\forall$ —

3.6 Riduzione di $\text{HALT}_\forall$ fuori da RE e coRE
Riduzione 1: $\text{HALT}_e ps \leqslant \text{HALT}_\forall$ — • Riduzione 2: $\overline{\text{HALT}_e ps} \leqslant \text{HALT}_\forall$ —

3.7 Teorema di Rice
Proprietà di macchina — • Proprietà semantica (di linguaggio) — • Teorema di Rice —

Chapter 4 **Complessità Strutturale** Page

4.1 Definizioni di Complessità Temporale
Notazione Asintotica — • Complessità di un Problema —

4.2 Classe DTime e la Classe P
Esempi in P —

4.3 Classe NTime e la Classe NP
SAT (Soddisfacibilità Booleana) — • Independent Set — • L'intuizione non deterministica —

4.4 La Questione P vs NP

4.5 Riduzioni Polinomiali

Chapter 5 **Complessita Spaziale** Page

Chapter 6 **Oracoli e Classi Funzionali** Page

Chapter 1

Introduzione a problemi e indecidibilita

Studieremo due arie:

- **Calcolabilita**: ci chiediamo se per quel problema esistera mai un algoritmo in grado di risolverlo (decidibilita' del problema). Non li incontriamo spesso irl dato che siamo piu' spinti a fare robe che sappiamo gestire. Bisogna studiare la struttura del problema per decidere se esiste o meno un algoritmo che lo risolve.
- **Complessita**: vogliamo determinare se un problema decidibile e' "facile" o "difficile", ovvero qual'e' la sua complessita' (diverso dalla complessita' degli algoritmi)

Note:

Il professore usa in modo informale i termini "facile" e "difficile" perche' si tratta solo di un'introduzione. Piu' avanti definiremo meglio questi concetti in base a se la complessita' e' *polinomiale* o meno.

Definition 1.0.1: Problema

Relazione fra stringhe.

Example 1.0.1 (Somma)

Dati x e y calcola $x + y \to z$. E' una relazione binaria fra stringhe:

- $< (2, 3), 5 >$
- $< (4, 3), 7 >$

  $\vdots$

Dove la prima stringa e' l'input e la seconda e' l'output. Dato che elencare tutte le tuple e' impossibile, le descriviamo a parole ma stando attenti ad essere precisi. Ci servono tre elementi:

- What is input?
- What is output?
- Che relazione c'e' fra out e in?

1.1 Halting Problem

Definiamo prima per bene il problema:

- Input: Una stringa P che rapresenta il codice sorgente di un programma
- Output: Un booleano
- Relazione: Se P termina ritorna T, altrimenti F

Sembra un problema semplice, ma vedremo che in realta' e' indecidibile. Per confermare cio' dobbiamo formulare una dimostrazione formale, usando quel che viene chiamato "metodo diagonale".
Il prof non va cosi' in dettaglio per il metodo usato nella dimostrazione (dato che e' solo un esempio introduttivo), ma secondo me e' interessante e fornisce un bell'esempio per l'utilita' della Teoria delle Categorie.

1.1.1 Il Metodo Diagonale

Il *metodo diagonale* e' uno schema utile per la dimostrazione di certi teoremi. E' stato ideato per la prima volta da Cantor per dimostrare, per esempio, che $\mathbb{R}$ non e' numerabile, dove viene usata la *diagonale* di una tabella per dimostrare un assurdo. Questo metodo e' stato successivamente usato per dimostrare altre proposizioni molto importanti, come il teorema di Cantor (quello sugli insiemi potenza), il paradosso di Russel e i teoremi dell'incompletezza di Godel.
La struttura della dimostrazione generale e' la seguente:

$$\begin{tikzcd}
 & \text{COSE} \times \text{COSE} \arrow[r, "X", "\text{potente}"'] & \text{VALORI} \arrow[d, "I", "\text{Inverti}"'] \\
\text{COSE} \arrow[ur, "D", "\text{Diagonale}"'] \arrow[rr, "S", "\text{Speciale}"'] & & \text{VALORI}
\end{tikzcd}$$

Lo schema e' molto astratto, e' formato solo da *punti* (i vertici) e *frecce* che possono essere composte per formare altre frecce. I primi possono essere insiemi e le frecce funzioni, come vedremo. Le quattro frecce hanno delle funzioni particolari:

- La funzione "potente" X: nella Teoria delle Categorie viene definita *morfismo punto-suriettivo*(?). E' la freccia che assumiamo esista per poi dimostrare l'assurdo.
- La funzione diagonale D: dato un elemento x ritorna la coppia (x, x). Raccoglie l'essenza del metodo diagonale in quanto e' la componente che causa l'*auto-referenzialita'* che sta al cuore della dimostrazione.
- La funzione che inverte I: dato l'output di X lo *inverte* in modo da causare l'assurdita'.
- La funzione "speciale" S: composizione delle tre frecce sopra, insieme all'assunzione iniziale permette di dimostrare l'assurdo.

Note:
Lowkey non l'ho spiegato benissimo dato che non ho studiato abbastanza le sue applicazioni e so solo le basi della Teoria delle Categorie quindi non e' rigoroso per niente. Forse nel futuro ci ritornero', boh.

Finita la costruzione, i prossimi passi sono:

1. Codifica S con una *COSA*, k.
2. Cosa succede se passo k a S?

Note:
In verita' non sono sicurissimo se questi passi fanno parte del metodo generale o solo dell'halting

1.1.2 Dimostrazione Halting Problem

Andiamo ora a vedere come puo' essere dimostrato che l'halting problem e' indecidibile.
Per prima cosa dobbiamo definire la controparte concreta delle frecce e punti dello schema:

- I punti *COSE* sono insiemi di tutti i programmi o dati possibili. Sfruttando la **Numerazione di Godel** possiamo attribuire a ognuno di questi un numero naturale. Quindi *COSE* diventa $\mathbb{N}$.

- I punti *VALORI* sono tutti i possibili output di un programma, quindi dei dati. Quindi *VALORI* diventa $\mathbb{N}$.
- La freccia X diventa il programma *HALT*, ovvero la funzione totale che prende un *Programma* P e un *Dato* D (entrambe $\in \mathbb{N}$) tale che:

$$X(P, D) = \begin{cases} \text{"termina"} & P \text{ termina su } D \\ \text{"diverge"} & P \text{ non termina su } D \end{cases}$$

  Per ora assumiamo l'esistenza di questo programma.
- La freccia D e' il programma che implementa la funzione totale $\mathbb{N} \to \mathbb{N} \times \mathbb{N}$

$$D(x) = (x, x)$$

  che e' facile da costruire e quindi esiste.

$$\begin{tikzcd}
\mathbb{N} \times \mathbb{N} \arrow[r, "X"] & \mathbb{N} \arrow[d, "I"] \\
\mathbb{N} \arrow[u, "D"] \arrow[r, "S"'] & \mathbb{N}
\end{tikzcd}$$

Non e' chiaro come dobbiamo definire I in modo da dimostrare l'assurdo, quindi lavoriamo top-down.
Dato che una composizione di programmi e' un programma, anche S e' sicuramente un programma, quindi una funzione parziale che puo' terminare o divergere dato un input.
Essendo S un programma, e' possibile rappresentarlo con un numero $k \in \mathbb{N}$. Analizziamo $S(k)$:

- $S(k)$ termina: in questo caso $X(k, k) =$ "termina", quindi $S(k) = I($"termina"$)$. Per rimanere coerente I deve terminare sull'input "termina".
- $S(k)$ non termina: in questo caso $X(k, k) =$ "diverge", quindi $S(k) = I("diverge")$. Per rimanere coerente I deve divergere sull'input "diverge".

Ma noi non vogliamo la coerenza, vogliamo costruire S in modo da dimostrare l'assurdo. Quindi definiamo I come la funzione parziale che:

- Diverge sull'input "termina"
- Termina sull'input "diverge"

Abbiamo quindi che:

- Se $X(k, k) =$ "termina", allora $S(k)$ diverge
- Se $X(k, k) = "diverge"$, allora $S(k)$ termina

Queste due implicazioni sono in opposizione alla definizione del programma *HALT*, che quindi non puo' esistere.

1.2 Tipi di problemi

Ci sono due categorie:

- **Ricerca**: vogliamo calcolare un risultato arbitrario, ovvero *ricercano* un output vario.
- **Decisione**: l'output e' *booleano*.

Queste due tipologie non sono completamente sganciate, esiste una relazione.
Dato un problema di ricerca, possiamo ricavare la versione "decisionale" che e' piu' facile da calcolare.

Example 1.2.1 (Somma)
Ricerca: dati a, b calcolare la somma $c = a + b$.
Decisione: dati a, b, c determinare se $c = a + b$ (vero o falso).

Per questo motivo parleremo quasi totalmente solo di problemi di decisione.

Note:
E' da qui che deriva il termine "decidibile".

1.2.1 Problemi di decisione

Theorem 1.2.1
Tutti i problemi di decisione possono essere ridotti al riconoscimento di uno specifico linguaggio.

Example 1.2.2
PATH $< G, s, t >$: vogliamo stabilire se esiste un percorso da s a t.
Mostriamo di poter trasformare questo problema decisionale in un problema che chiede se una "parola" appartiene a uno specifico linguaggio.

$$L_{\text{PATH}} = \{< G, s, t > | G \text{ e' un Grafo } \wedge s, t \text{ sono nodi del grafo } G \wedge \exists \text{ un percorso da } s \rightarrow t\}$$

Ci riduciamo quindi a decidere se una data tripla appartiene a L_{PATH} o meno.

Quindi, mentre studiamo decidibilita' dei problemi guarderemo linguaggi e i loro *automi*.

1.3 Linguaggi

1.3.1 Definizioni

Definition 1.3.1: Alfabeto

$\Sigma = \{a, b, c, ...\}$ e' l'*alfabeto*
E' un insieme di simboli.

Definition 1.3.2: Parola

w e' una parola o striga su Σ e' un concatenamento di 0 o piu' simboli di Σ.

Definition 1.3.3: Kleene

def

Definition 1.3.4: Linguaggio

Sottoinsieme delle parole che si possono costruire su un determinato alfabeto Σ:

$$L \subseteq \Sigma^*$$

Definition 1.3.5

Dato un linguaggio L **fissato** per una stringa w, decidere se:

$$w \in L$$

Il problema di appartenenza a un linguaggio L (**fissato!**) e' definito come:
dato il linguaggio L, prendere in input una parola w e decidere se $w \in L$.
input: stringa output: booleano
Perche' vogliamo usare automi? : e' un formalismo piu' basico e piu' semplice da studiare rispetto a un arbitrario linguaggio di prorammazione.

1.3.2 Automi

Vabbe' introduce gli automi, il Guerriero ci ha gia preparato adeguatamente.
Alright whatsapp.

1.3.3 Macchia di Turing

Automa piu' potente che Turing si invento per studiare i problemi importanti da chiudere, fra cui l'automatizzazione dei teoremi.
E' nato per formalizzare il calcolo automatico in un tempo in cui non esistevano i computer, era la versione "automatizzata" di lui che scriveva su carta.
Iniziamo con una spiegazione informale:
E' una macchina caratterizzata da un nastro con delle cellette
In ognuna cella c'e' un simbolo, ed e' infinito in entrambe le direzioni
Ha una testina posizionata su una cella da cui puo' leggere il simbolo
La testina puo' spostarsi a dx o sx, e a differenza degli automi visti puo' benissio tornare indietro e cambiare la decisione presa prima (backtracking)
La testina puo' anche scrivere su una cella, perdendo cio' che c'era prima (una seconda differenza)
Per noi sta roba e' un algoritmo, dato che la macchina di Turing, in quanto automa, ha vari stati di funzionamento

- Legge simbolo cella
- In base allo stato, sovrascrive qualcosa
- Sposto la testina avanti o indietro
- Cambio lo stato

Example 1.3.1

Progetta una MdT che riconosca il linguaggio

$$L = \{a^m b^m | m \geqslant 0\}$$

La stringa da decidere si trova sul nastro da sx a dx e la testina si trova sopra il primo simbolo.
Un algoritmo e' un filmato di quello che succede nella nostra testa per risolvere un problema.

- Leggi cella, se e' vuota fine (ok)
- altrimenti se e' b fine (no)
- se e' a, cancella
- spostati a dx finche' la cella e' bianca
- vai a sx di uno e leggi
- se e' a, fine (no)
- se e' b, cancella e vai a sx finche e' bianca
- muoviti di uno a dx

TODO: disegna automa

Definizioni formali

Definition 1.3.6: Macchina di Turing

E' la settupla

$$M = \{Q, q_0, F, \Sigma, \Gamma, b/, \delta\}$$

Dove:

- Q e' l'insieme di stati
- q_0 e' lo stato iniziale
- F e' l'insieme degli stati finali
- Σ e' l'insime dei simboli in input
- Γ e' l'insieme dei simboli che la macchina puo' scrivere $\Sigma \subset \Gamma$
- $b/$ e' il simbolo di cella vuota ("blank") $b/\in \Gamma$
- δ e' la funzione

$$Q \times \Gamma \to Q \times \Gamma \times \{\leftarrow, \rightarrow\}$$

La macchina di turing ha stati finiti (come gli altri automi) e il suo programma e' fissato (non cambia). Quindi come fa a eseguire qualunque computazione? Vedremo.
δ codifica le funzioni di transizioni (gli archi dell'automa)

Definition 1.3.7: Configurazione

Sia $M = \{Q, q_0, F, \Sigma, \Gamma, \not{b}, \delta\}$ una MdT. Si definisce *configurazione* una stringa $\alpha q \beta$ dove

- $q \in Q$ è lo stato corrente della macchina.
- $\alpha, \beta \in \Gamma^*$ sono le stringhe di simboli che si trovano a sinistra e a destra della testina.

Note:

Si può esprimere informalmente come una "fotografia" dello stato corrente di computazione della macchina:

- Stato macchina
- Posizione testina
- Cosa c'e' sul nastro

Example 1.3.2

$$aq_1abb$$

dove q_1 e' lo stato della macchina e la testina legge il carattere a dx dello stato.

$$aabq_3b/$$

sta leggendo la prima cella bianca a dx
Nota: i caratteri bianchi ci permettono di precisare la posizione della testina, ma non ne dobbiamo mettere infiniti obv

Partendo dalla configurazione, usando la funzione di transizione siamo in grado di calcolare il passo che eseguira'

Definition 1.3.8: Configurazione Accettante

Quando al suo interno si trova uno stato accettante

Definition 1.3.9: Legal Successor

Data una MdT M e sia c_1 una sua configurazione, c_2 si dice *legal successor* di c_1 rispetto a M se c_2 si ottiene da c_1 in esattamente un passo di computazione mediante l'applicazione di una transizione definita nella funzione di transizione δ.
E lo si denota con:

$$c_1 \vdash_M c_2$$

Example 1.3.3 (Legal successor)
Sia M una MdT e sia $c_1 = aq_1b$ una sua configurazione. Se $\delta(q_1, b) = (q_2, a, \rightarrow)$, allora $c_2 = aaq_2b$ e' legal successor di c_1 rispetto a M

Definition 1.3.10: Configurazione iniziale

si definisce configurazione iniziale di una MdT M e di una stringa $w \in \Sigma^*$ la configurazione $c_1 = q_0w$

Definition 1.3.11: Configurazione Finale

Configurazione che non ammette legal successor secondo M

Definition 1.3.12: Computazione Parziale

Data una MdT M, una *computazione parziale* per M e' una sequenza:

$$c_1, c_2, \ldots, c_n$$

dove

$$c_1 \vdash_M c_2 \vdash_M \cdots \vdash_M c_n$$

Note:
Non è detto che c_1 sia il punto di partenza e c_n sia l'ultima configurazione della computazione, per questo è parziale.

Definition 1.3.13: Computazione

Data una MdT M e una stringa $w \in \Sigma^*$, una *computazione* per M e' una computazione parziale $c_1, c_2, \ldots, c_n$ per M dove c_1 è una configurazione iniziale e c_n è una configurazione finale

Possiamo ora definire quando una MdT accetta un input

Definition 1.3.14: Accettazione

Una MdT accetta w se si arresta su uno stato accettante, e rifiuta w se non si arresta su uno stato accettante.

Note:
Da questa definizione, se una MdT non si arresta allora l'input non e' accettato. Quindi per non accettare puo' anche divergere. Questa roba qua per noi e' un po' cringe.

Definition 1.3.15: Linguaggio di una MdT

Linguaggio

$$\mathcal{L}(M) = \{w | M(w) = 1\}$$

Definition 1.3.16: Decide

Sia L un linguaggio, M *decide* L sse $\forall w \in \Sigma^*$

$$w \in L \implies M \text{ accetta } w$$

$$w \notin L \implies M \textit{ si arresta } \text{e dice di "no" } w$$

Definition 1.3.17: Riconosce

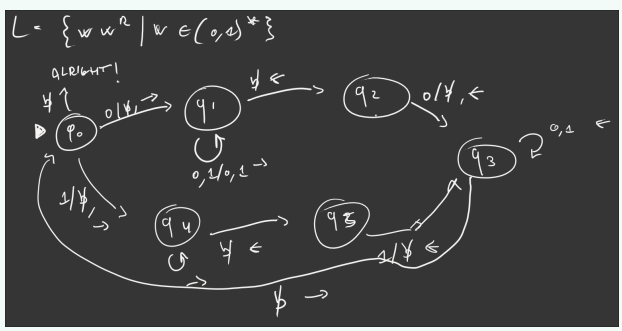
Sia L un linguaggio, M *riconosce* L sse $\forall w \in \Sigma^*$

$$w \in L \implies M \text{ accetta } w$$

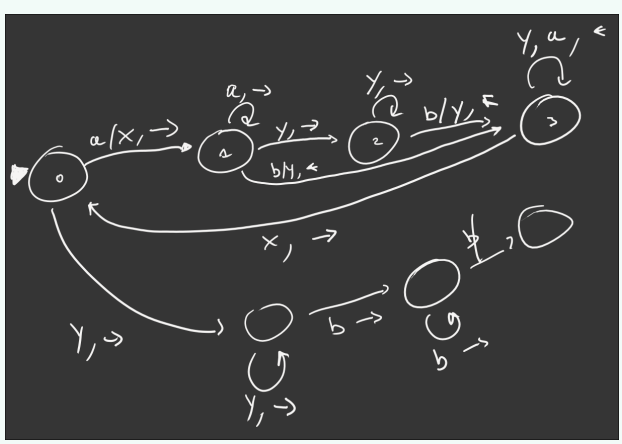
$$w \notin L \implies M \text{ non accetta } w$$

Esercizi

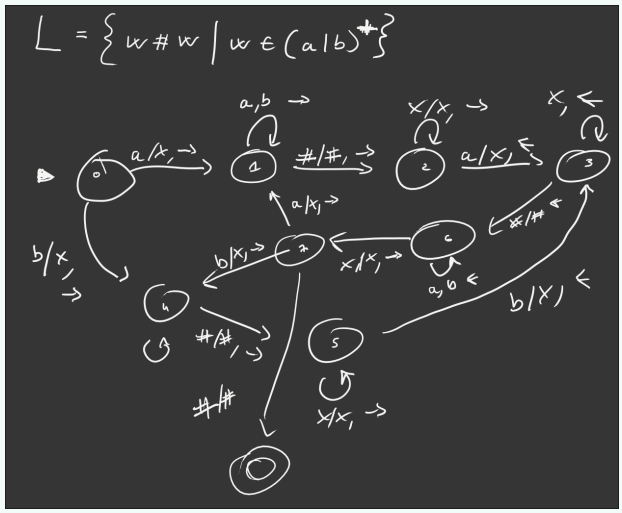
Example 1.3.4



Example 1.3.5



Example 1.3.6



Chapter 2

Macchine di Turing e Linguaggi

2.1 Linguaggi

2.1.1 Definizioni

Definition 2.1.1: Alfabeto

$\Sigma = \{a, b, c, ...\}$ e' l'*alfabeto*
E' un insieme di simboli.

Definition 2.1.2: Parola

w e' una parola o striga su Σ e' un concatenamento di 0 o piu' simboli di Σ.

Definition 2.1.3: Kleene

def

Definition 2.1.4: Linguaggio

Sottoinsieme delle parole che si possono costruire su un determinato alfabeto Σ:

$$L \subseteq \Sigma^*$$

Definition 2.1.5

Dato un linguaggio L **fissato** per una stringa w, decidere se:

$$w \in L$$

Il problema di appartenenza a un linguaggio L (**fissato!**) e' definito come:
dato il linguaggio L, prendere in input una parola w e decidere se $w \in L$.
input: stringa output: booleano
Perche' vogliamo usare automi? : e' un formalismo piu' basico e piu' semplice da studiare rispetto a un arbitrario linguaggio di prorammazione.

2.1.2 Automi

Vabbe' introduce gli automi, il Guerriero ci ha gia preparato adeguatamente.
Alright whatsapp.

2.2 Macchia di Turing

Automa piu' potente che Turing si invento per studiare i problemi importanti da chiudere, fra cui l'automatizzazione dei teoremi.
E' nato per formalizzare il calcolo automatico in un tempo in cui non esistevano i computer, era la versione "automatizzata" di lui che scriveva su carta.
Iniziamo con una spiegazione informale:
E' una macchina caratterizzata da un nastro con delle cellette
In ognuna cella c'e' un simbolo, ed e' infinito in entrambe le direzioni
Ha una testina posizionata su una cella da cui puo' leggere il simbolo
La testina puo' spostarsi a dx o sx, e a differenza degli automi visti puo' benissio tornare indietro e cambiare la decisione presa prima (backtracking)
La testina puo' anche scrivere su una cella, perdendo cio' che c'era prima (una seconda differenza)
Per noi sta roba e' un algoritmo, dato che la macchina di Turing, in quanto automa, ha vari stati di funzionamento

- Legge simbolo cella
- In base allo stato, sovrascrive qualcosa
- Sposto la testina avanti o indietro
- Cambio lo stato

Example 2.2.1

Progetta una MdT che riconosca il linguaggio

$$L = \{a^m b^m | m \geqslant 0\}$$

La stringa da decidere si trova sul nastro da sx a dx e la testina si trova sopra il primo simbolo.
Un algoritmo e' un filmato di quello che succede nella nostra testa per risolvere un problema.

- Leggi cella, se e' vuota fine (ok)
- altrimenti se e' b fine (no)
- se e' a, cancella
- spostati a dx finche' la cella e' bianca
- vai a sx di uno e leggi
- se e' a, fine (no)
- se e' b, cancella e vai a sx finche e' bianca
- muoviti di uno a dx

TODO: disegna automa

2.2.1 Definizioni formali

Definition 2.2.1: Macchina di Turing

E' la settupla

$$M = \{Q, q_0, F, \Sigma, \Gamma, b/, \delta\}$$

Dove:

- Q e' l'insieme di stati
- q_0 e' lo stato iniziale
- F e' l'insieme degli stati finali
- Σ e' l'insime dei simboli in input
- Γ e' l'insieme dei simboli che la macchina puo' scrivere $\Sigma \subset \Gamma$
- $b/$ e' il simbolo di cella vuota ("blank") $b/\in \Gamma$
- δ e' la funzione

$$Q \times \Gamma \to Q \times \Gamma \times \{\leftarrow, \rightarrow\}$$

La macchina di turing ha stati finiti (come gli altri automi) e il suo programma e' fissato (non cambia). Quindi come fa a eseguire qualunque computazione? Vedremo.
δ codifica le funzioni di transizioni (gli archi dell'automa)

Definition 2.2.2: Configurazione

E' una "fotografia" dello stato corrente di computazione della macchina:

- Stato macchina
- Posizione testina
- Cosa c'e' sul nastro

Example 2.2.2

$$aq_1abb$$

dove q_1 e' lo stato della macchina e la testina legge il carattere a dx dello stato.

$$aabq_3b/$$

sta leggendo la prima cella bianca a dx
Nota: i caratteri bianchi ci permettono di precisare la posizione della testina, ma non ne dobbiamo mettere infiniti obv

Partendo dalla configurazione, usando la funzione di transizione siamo in grado di calcolare il passo che eseguira'

Definition 2.2.3: Configurazione Accettante

Quando al suo interno si trova uno stato accettante

Definition 2.2.4: Legal Successor

Data una MdT M e sia c_1 una sua configurazione, c_2 si dice *legal successor* di c_1 sse:

$$c_1 \vdash_M c_2$$

Definition 2.2.5: Configurazione Finale

Configurazione che non ha legal successor

Definition 2.2.6: Computazione Parziale

Data una MdT M, una *computazione parziale* per M e' una sequenza:

$$c_1, c_2, ...c_n$$

dove

$$c_1 \vdash_M c_2 \vdash_M ... \vdash_M c_n$$

Note:

Non e' detto che c_1 sia il punto di partenza e c_n sia l'ultima configurazione della configurazione, per questo e' parziale.

Definition 2.2.7: Computazione

Data una MdT M e una stringa $w = w_1 w_2 ... w_l$, una *computazione* per M e' una computazione parziale per M dove

$$c_1 = q_0 w_1 w_2 ... w_l$$

e c_n e' una configurazione finale

Possiamo ora definire quando una MdT accetta un input

Definition 2.2.8: Accettazione

Una MdT accetta w se si arresta su uno stato accettante, e rifuita w se non e' vero che si arresti in uno stato accettante.

Note:

Da questa definizione, se una MdT non si arresta allora l'input non e' accettato. Quindi per non accettare puo' anche divergere. Questa roba qua per noi e' un po' cringe.

Definition 2.2.9: Linguaggio di una MdT

Linguaggio

$$\mathcal{L}(M) = \{w | M(w) = 1\}$$

Definition 2.2.10: Decide

Sia L un linguaggio, M *decide* L se $\forall w \in \Sigma^*$

$$w \in L \implies M \text{ accetta } w$$

$$w \notin L \implies M \textit{ si arresta} \text{ e dice di "no" } w$$

Definition 2.2.11: Riconosce

Sia L un linguaggio, M *riconosce* L se $\forall w \in \Sigma^*$

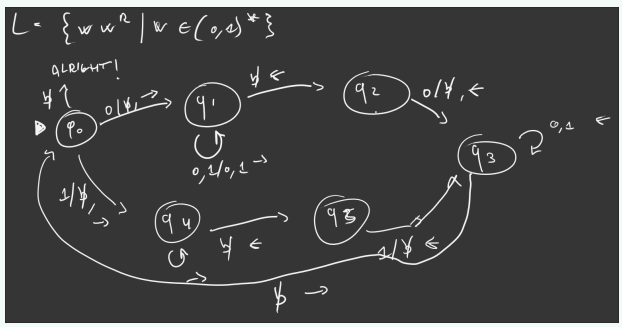
$$w \in L \implies M \text{ accetta } w$$

$$w \notin L \implies M \text{ non accetta } w$$

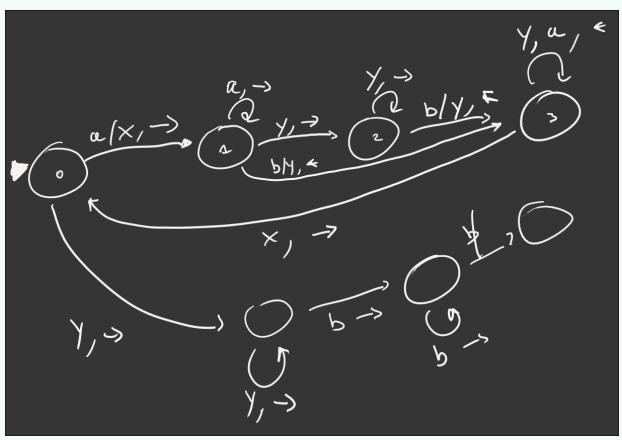
2.2.2 Esercizi

TODO: sarebbe figo fare gli automi in latex, se ho sbatti ci provo. Ah e alcune soluzioni sono imprecise le dovrei fixare.

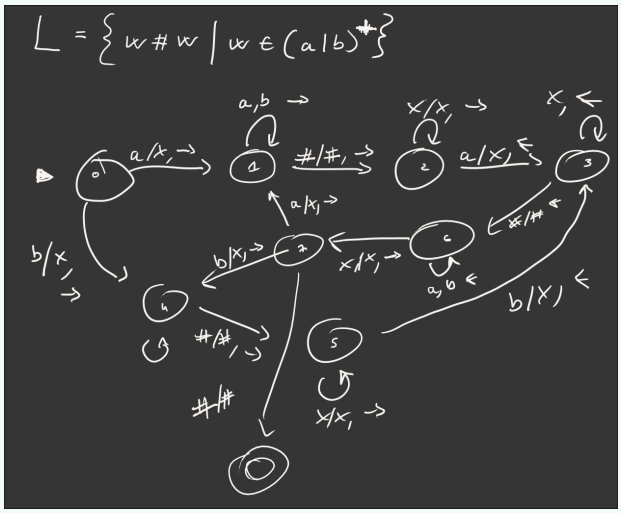
Example 2.2.3



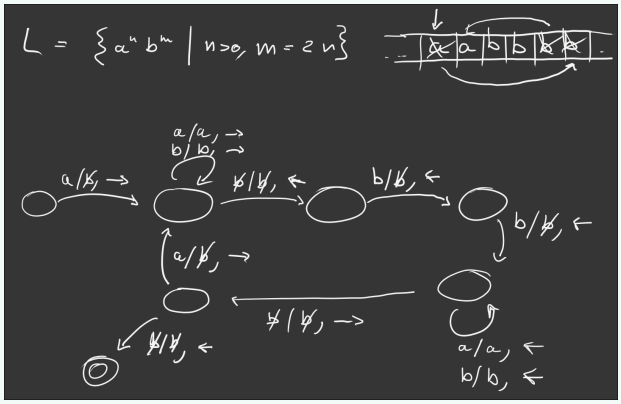
Example 2.2.4



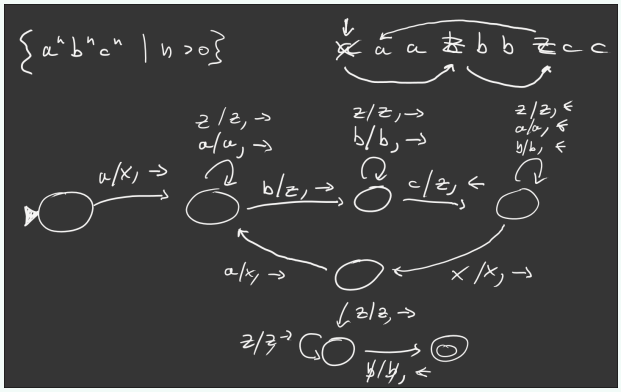
Example 2.2.5



Example 2.2.6



Example 2.2.7



2.3

Le macchine che abbiamo visto finora sono molto tediose.
Per esempio, per riconoscere il linguaggio $L = \{ww^R | w \in (0|1)^*\}$ e' stato necessario costruire due rami (uno per lo 0 e uno per l'1). Sarebbe comodo poter usare un solo ramo che in qualche modo codifica entrambe i rami,

vediamo come possiamo fare:

Definition 2.3.1: Memoria nello stato

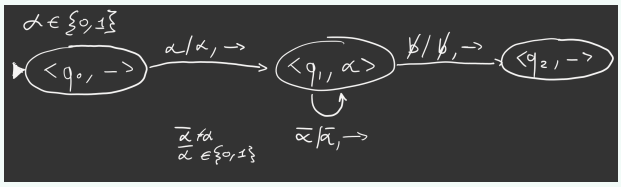
Data una MdT, si dice che questa ha **memoria** (nello stato) se ad ogni stato e' associata una memoria di uno o piu' simboli:

$$\langle q_1, -, ..., - \rangle$$

Il numero di simboli in memoria e' fissato, ovvero non puo' cambiare durante l'esecuzione.

Example 2.3.1

Vogliamo riconoscere $L = 10^*|01^*$



Notiamo che la macchina con memoria sopra non e' altro che una rappresentazione piu' compatta di una normale MdT con due rami al posto di uno.

Proposition 2.3.1 Equipotenza macchine con memoria

Data una MdT con memoria nello stato, e' sempre possibile costruire una MdT senza memoria che riconosce lo stesso linguaggio.

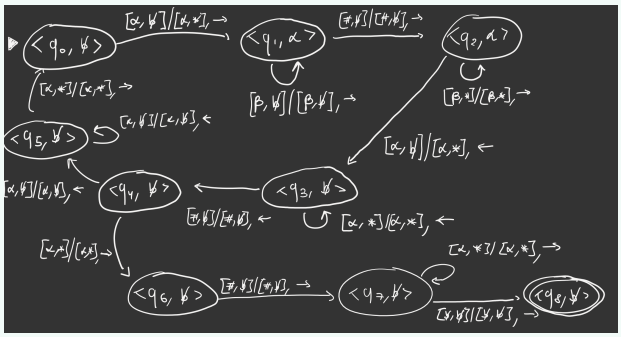
Dimostrazione: Boh in verita' non dimostra sto risultato, dice solo che alla fine le macchine con memoria sono un prodotto cartesiano di stati con simboli.

Definition 2.3.2: Macchine Multitraccia

Multiple tracce lette in sincrono, la testina si muove tutta insieme ma possiamo leggere e scrivere singolarmente sulle tracce.

Example 2.3.2

Vogliamo riconoscere $L = \{w\#w | w \in (a|b)^+\}$:



Proposition 2.3.2 Equipotenza macchine multitraccia

Data una macchina multitraccia, e' sempre possibile costruire una MdT classica che riconosce lo stesso linguaggio.

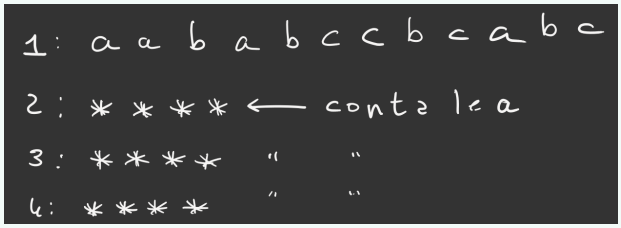
Dimostrazione: Data una macchina multitraccia di n tracce con alfabeti Γ_i per ogni traccia i, possiamo costruire una MdT con $\Gamma = \Gamma_1 \times ... \times \Gamma_n$ che si comporta esattamente come la macchina multitraccia.

Definition 2.3.3: Macchine Multinastro

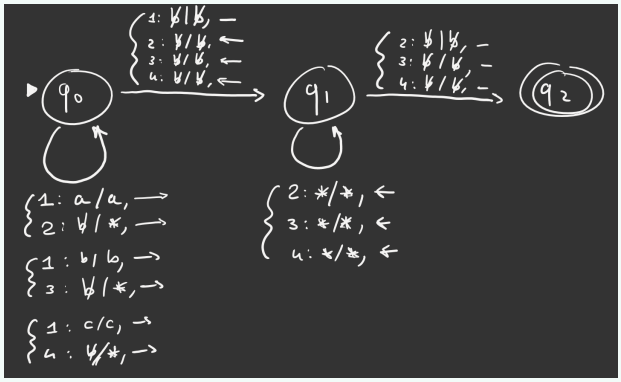
MdT che ha piu' nastri con testine indipendenti. Il numero dei nastri e' fissato a costruzione. E' possibile che una testina rimanga ferma.

Example 2.3.3

Costruiamo la macchina multinastro che riconosce $L = \{w | w \in (a|b|c)^* \wedge$ il numero delle a, b e c sono uguali$\}$. Vediamo l'idea:



Ora costruiamo l'automa:



Proposition 2.3.3 Equipotenza di macchine multinastro

Data una macchina multinastro, e' sempre possibile costruire una MdT classica che riconosce lo stesso linguaggio.

Dimostrazione: Data una macchina multinastro e' possibile costruire una multitraccia equivalente con $2n$ tracce (dove n e' il numero dei nastri) semplicemente usando le tracce dispari come "puntatori" che indicano l'ipotetica posizione della testina, come se fosse multinastro.
Ad ogni passo, collochiamo inizialmente la testina sul primo puntatore a sx per poi scansionare tutto fino all'ultimo puntatore. I simboli sotto ogni puntatore vengono salvati nella memoria, e la testina ritorna indietro cambiando in modo adeguato i simboli puntati e la posizione dei puntatori.

E' il modello di calcolo piu' flessibile che abbiamo a disposizione. Ma abbiamo visto che per simulare il suo comportamento con una macchina multitraccia impieghiamo piu' transizioni, ma quante di piu'?

Dimostrazione: Analizziamo il caso peggiore dopo m passi. Questo avviene quando una testina della macchina si sposta a sx ad ogni passo e una si sposta a dx ad ogni passo, in modo che distino $2m$ celle.
Quindi una macchina multinastro avra' bisogno di percorrere due volte la distanza (passaggio in avanti e indietro) e deve poi aggiornare ogni puntatore (uno per ogni nastro), che nel caso peggiore comporta altre due transizioni. Alla fine abbiamo che un solo passo di una macchina multinastro (k nastri) richiede nel caso peggiore $4m + 2k$ passi di una macchina multitraccia.

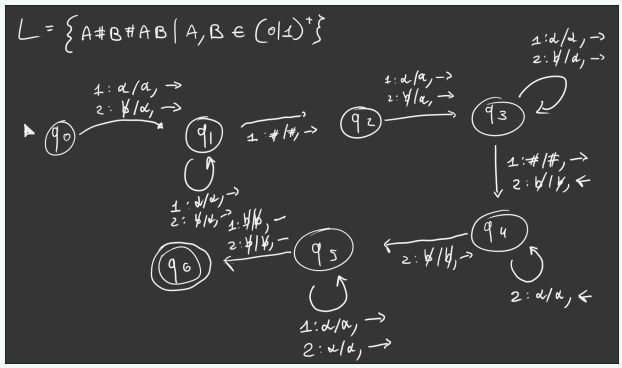
Possiamo quindi dire che una computazione che richiede n passi ad una macchina multinastro richiedera'

$$\sum_{i=1}^{n} 4i + 2k \leqslant \sum_{i=1}^{n} 4n + 2k \leqslant n(4n + 2k)$$

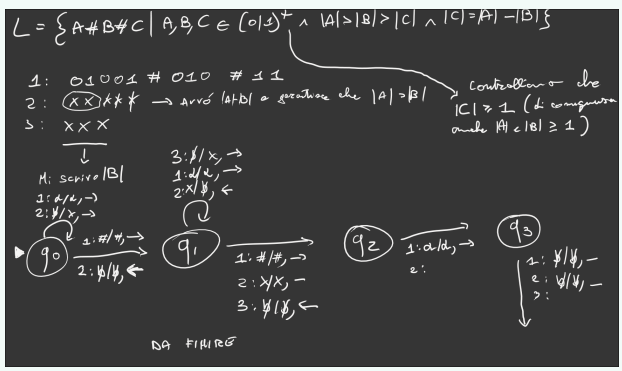
Quindi e' $O(n^2)$.

2.3.1 Esercizi

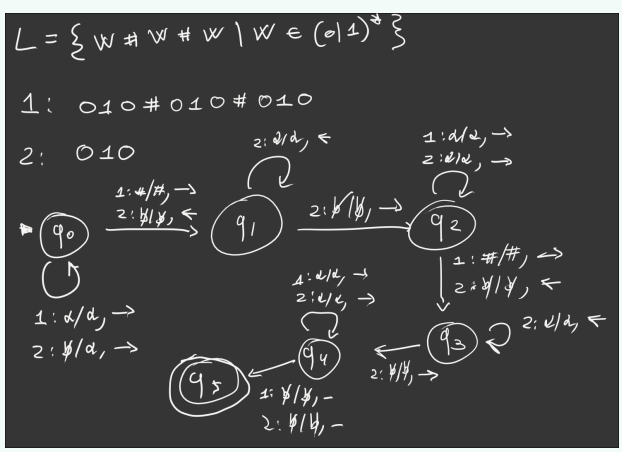
Example 2.3.4



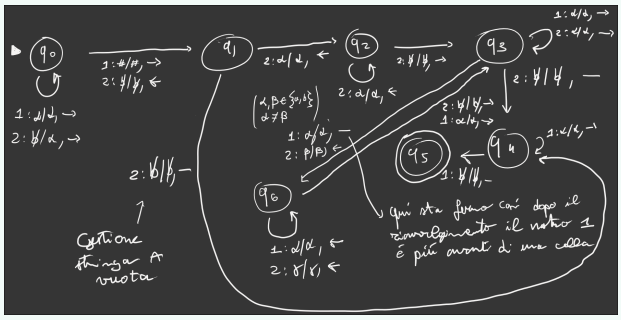
Example 2.3.5



Example 2.3.6



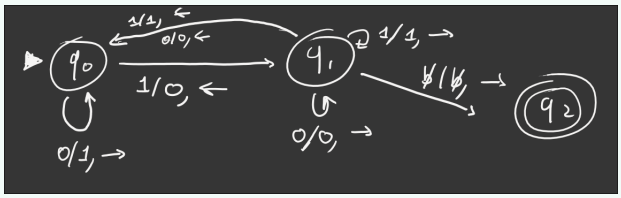
Example 2.3.7



2.3.2 Macchine Non Deterministiche

In verita' adesso introduciamo un altro tipo di macchina crazy che ci servira', vediamo un esempio:

Example 2.3.8



Notiamo che e' una macchina particolare perche' la sua funzione di transizione specifica un insieme di possibili passi data una configurazione.

Definition 2.3.4: Macchina Non-deterministica

N e' una macchina non-deterministica se e':

$$N = \langle \Sigma, \Gamma, b\, Q, q_0, F, \delta \rangle$$

dove δ cambia rispetto alla classica macchina:

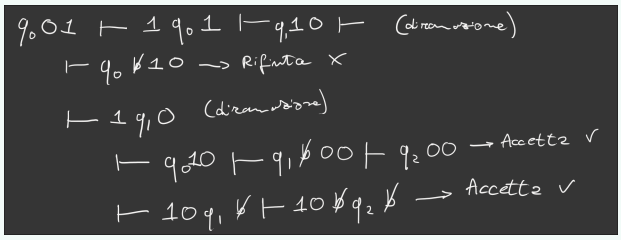
$$\delta : Q \times \Gamma \to 2^{Q \times \Gamma \times \{\leftarrow, \rightarrow\}}$$

Note:

Le macchine deterministiche sono un sottoinsieme degenere delle macchine non deterministiche dove l'immagine e' formata solo da insiemi di cardinalita' 1.

Example 2.3.9

Supponiamo di voler vedere come funziona la macchina di prima su un input 01:



Ma accetta o rifuita?

Supponiamo di avere una bella MdT non-deterministica, cosa darebbe in output? Questa e' una domanda a trabocchetto perche' in verita' non potrebbe esistere (le macchine quantistiche sono meno potenti). Come vogliamo definire se una stringa e' accettata quindi?
Ci possiamo studiare l'*albero di computazione*, che usa le configurazioni (che hanno significato analogo a quelle deterministiche) con le stesse definizioni di stato iniziale, finale e accettante, pero' cambiano i legal succesors.
Quindi i nodi sono tutte le configurazioni che una macchina non deterministica puo' provare processando la stringa w. La radice e' la config iniziale, e abbiamo un arco fra configurazioni $a \to b$ se la config b e' legal successor della config a. Quindi la computazione e' un albero e non una lista.
La stringa w in input viene accettata quando nell'albero di computazione esiste almeno un percorso dalla radice a uno stato accettante.
Ma sta roba e' capace di riconoscere linguaggi che le macchine deterministiche non sono in grado di descrivere?

Proposition 2.3.4

Una macchina di turing non deterministica ha lo stesso potere espressivo di quelle deterministiche.

: La macchina non determinista N puo' essere ricondotta a una determinista M

$$N \longrightarrow M \ \mathcal{L}(N) = \mathcal{L}(M)$$

Immaginiamo che N sia mononastro e M binastro.
Stiamo esplorando l'albero in BFS dato che, se ci fosse una computazione infinita nell'albero e lo esploriamo in DFS, e' possibile che la macchina deterministica si "blocchi" a esplorare il ramo infinito quando forse puo' esserci un'altro ramo che in realta' e' accettante.

Ma questa simulazione quanto costa?
Supponiamo che il numero massimo di figli nell'albero di computazione abbia al massimo c figli, allora in numero di nodi totali di un albero di altezza m e'

$$O(e^m)$$

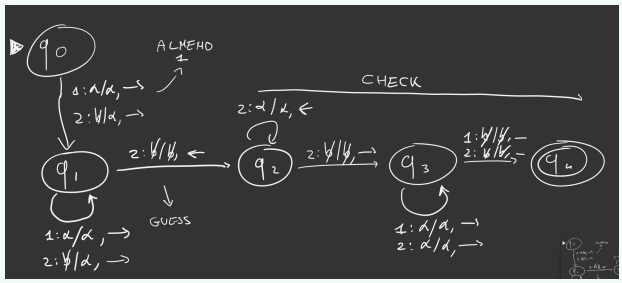
Quindi anche il costo per simulare la BFS di questo albero sara' esponenziale. Ma in realta' non siamo sicuri che questo sia il limite, e non siamo neanche stati in grado di dimostrare che questa cosa non e' possibile. P vs NP e' incentrata proprio su questa questione
Ipotesi di Church-Turing:
Tutto cio' che e' calcolabile e' calcolabile da una macchina di Turing
E' self referential, non abbiamo una definizione di calcolabile, semplicemente non abbiamo sviluppato altri formalismi di calcolo con potere espressivo piu' alto. Quindi appunto non e' un teorema
Introduciamo le prime due classi di calcolabilita':

- **RE**: ci sono tutti i linguaggi semi-decidibili, ovvero per i quali esiste una MdT che accetta (non decide) il linguaggio (potrebbe non terminare).
- **R**: inclusa in RE, classe dei problemi di decisione ricorsivi (linguaggi ricorsivi), ovvero per i quali esiste una MdT che quindi da' garanzia di risposta. Sono quindi linguaggi decidibili.

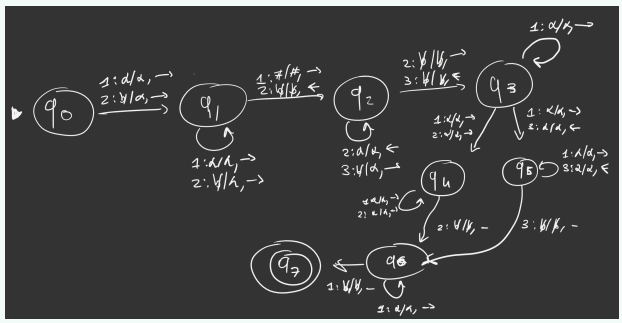
Definition 2.3.5: Classe

Semplicemente un insieme di linguaggi.

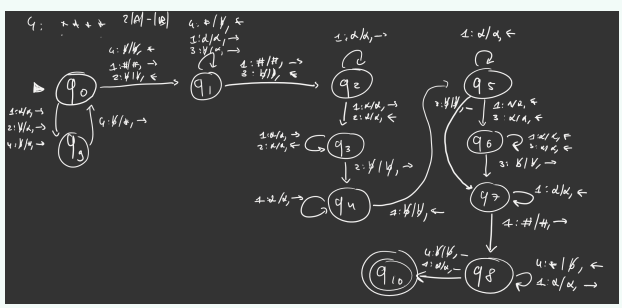
Example 2.3.10



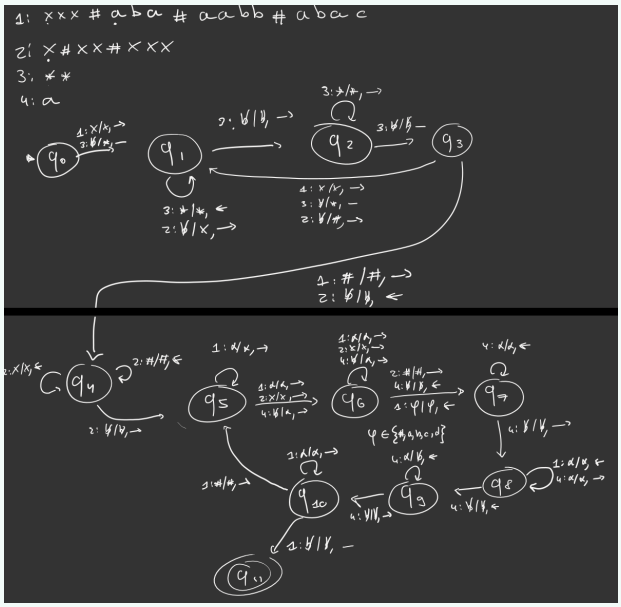
Example 2.3.11



Example 2.3.12



Example 2.3.13



Ha spammato degli esercizi, onesto possiamo anche guardarli da virtuale.
Abbiamo introdotto le macchine nondeterministiche per affrontare i problemi in modo piu' semplice, ora che siamo forti useremo algoritmi nondeterministici per risolvere problemi in modo da dimostrare la loro decidibilita'.

2.4 Macchine di turing universali

Le macchine di turing presentate finora sono "statiche" ovvero accettano/decidono un solo linguaggio. Sono programmate solo per fare una cosa, dato che la funzione di transizione è fissata.
Tuttavia i computer che usiamo tutti i giorni sono "dinamici" ovvero possono eseguire qualunque programma, ovvero la funzione di transizione è più "ricca" di informazioni, dato che deve codificare anche il programma che deve eseguire.
Quindi ci poniamo il problema: è possibile costruire una macchina di turing che possa eseguire qualunque programma? La risposta è sì, ed è la macchina di turing universale.

2.4.1 Codifica della macchina di turing

Consideriamo una MdT il cui alfabeto di input abbia solo due simboli $\Sigma = \{0, 1\}$, mentre l'alfabeto di nastro può essere $\Gamma = \{0, 1, \not{b}, \dots\}$.
La funzione di transizione ha generalmente la forma:

$$\delta(Q_i, X_j) = (Q_k, X_l, D_m)$$

Example 2.4.1

$$\delta(q_3, \not{b}) = (q_5, 0, \rightarrow)$$

Codifichiamo i simboli Q_i, X_j, Q_k, X_l, D_m partendo dai loro pedici:

$$Q_i = 0^i, \quad X_j = 0^j, \quad \text{sx} = D_1, \quad \text{dx} = D_2$$

Note:

Per convenzione fissiamo:

- $x_1 = 0,\ x_2 = 1,\ x_3 = \not{b},\ x_4, \dots$
- q_1 è sempre lo stato iniziale

- q_2 è sempre il singolo stato accettante

Ogni singola transizione c_i viene quindi codificata come una stringa binaria in cui i vari campi sono separati da 1. Ad esempio, la transizione $\delta(q_3, \not{b}) = (q_5, x_1, D_2)$ viene codificata come:

$$c_1 = \underbrace{000}_{q_3}\ 1\ \underbrace{000}_{X_3}\ 1\underbrace{00000}_{q_5}1\ \underbrace{0}_{X_1}\ 1\ \underbrace{00}_{D_2}$$

Per unire più transizioni in un'unica stringa, le istanze c_i vengono separate da 11:

$$c_1 11 c_2 11 c_3 11 c_4 \cdots$$

Note:

Esistono infiniti modi per codificare una MdT, ma scegliamo questo. L'obiettivo è poter comunicare a una MdT come comportarsi rispetto a un'altra, quindi ogni stringa binaria può essere messa in associazione con una possibile MdT.
Ci saranno inevitabilmente stringhe binarie che non codificano nessuna MdT valida (perché non rispettano il formato). Per convenzione, assumiamo che tali stringhe corrispondano a una macchina con un unico stato non accettante.

2.4.2 Definizione formale

Definition 2.4.1: Macchina di Trueing universale

La Macchina di Turing Universale M_u e' una macchina che riceve in input una stringa binaria che codifica una macchina M e poi una stringa w

$$(M, w)$$

La macchina M_u e' in grado di dirci se M avrebbe accettato o meno w

Definition 2.4.2: Linguaggio universale

Linugaggio che accetta una macchina universale M_u

$$L_u = \{\langle M, w\rangle | M \models w\}$$

2.4.3 Processo di simulazione

La macchina M_u è una macchina a **quattro nastri**:

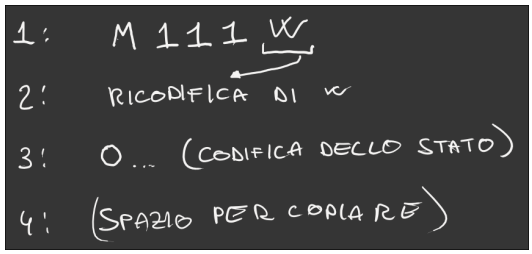
- **Nastro 1 (input)**: contiene la coppia M `111` w, dove M e w sono separati da tre 1 consecutivi. Questo separatore è non ambiguo poiché nella codifica di M non compaiono mai tre 1 di fila.
- **Nastro 2 (nastro simulato)**: contiene la ricodifica di w secondo i simboli simulati.

  $$C(w_i) = \begin{cases} 0 & w_i = 0 \\ 00 & w_i = 1 \end{cases}$$

  e la codifica completa

  $$C(w_1)1C(w_2)1...C(w_n)$$

- **Nastro 3 (stato simulato)**: tiene traccia dello stato corrente di M durante la simulazione.
- **Nastro 4 (lavoro)**: nastro ausiliario per operazioni di copia e spostamento.



Se la codifica in input non e' sensata, allora la macchina universale simula la macchina che rifiuta tutto. Altrimenti la simulazione procede nelle seguenti fasi:

1. **Inizializzazione**: M_u ricodifica w sul nastro 2 secondo lo schema dei simboli simulati. Ad esempio:

$$w = \underbrace{0}_{X_1}\ \underbrace{1}_{X_2}\ \underbrace{0}_{X_1} \implies \texttt{1 0 1 0 1}$$

   Scrive sul nastro 3 lo stato iniziale $q_1 = 0$.

2. **Ciclo di simulazione**: ad ogni passo, M_u conosce lo stato corrente di M (nastro 3) e il simbolo correntemente letto (nastro 2). Cerca quindi sul nastro 1 la transizione corrispondente $\delta(q_i, x_j)$ e:
   - aggiorna il nastro 3 con il nuovo stato q_k
   - sovrascrive il simbolo sul nastro 2 con x_l
   - sposta la testina sul nastro 2 nella direzione D_m
   - usa il nastro 4 come area di lavoro se è necessario ridimensionare il nastro 2

3. **Terminazione**: la simulazione termina quando M raggiunge una configurazione finale:

$$M \text{ accetta } w \implies M_u \text{ accetta}$$
$$M \text{ rifiuta } w \implies M_u \text{ rifiuta}$$
$$M \text{ non termina su } w \implies M_u \text{ non termina}$$

Note:

I nostri computer sono esattamente delle M_u. La loro funzione di transizione universale è fissata nell'hardware: è il **microcodice della CPU**. Questo risponde alla domanda su come mai le MdT classiche non si possono "programmare" mentre i computer sì.

2.5 Problemi Ricorsivi e Ricorsivamente Enumerabili

Abbiamo introdotto le classi di calcolabilità R e RE, e abbiamo detto che i problemi possono stare in R e in RE. Ma possono esistere problemi che stanno fuori da RE? Sì, e ora li andremo a vedere.

2.5.1 Enumerare stringhe e macchine

Riconsideriamo macchine che processano in input stringhe binarie. Se prendiamo una stringa binaria e ci mettiamo un 1 davanti, possiamo associarle un numero intero:

$$1 \circ 00010\ldots$$

In questo modo possiamo enumerare tutte le stringhe binarie:

$$\underbrace{1 \circ \varepsilon}_{1}\quad \underbrace{1 \circ 0}_{2}\quad \underbrace{1 \circ 1}_{3}\quad \underbrace{1 \circ 01}_{4}\quad \underbrace{1 \circ 10}_{5}\quad \cdots$$

Le stringhe con indice più basso sono tendenzialmente più corte; stringhe della stessa lunghezza sono ordinate lessicograficamente. Le stringhe binarie sono **infinite numerabili**.
Lo stesso ragionamento vale per le MdT: grazie alla codifica binaria introdotta nella sezione precedente, possiamo mettere un 1 davanti alla codifica di ogni MdT e ottenere un numero intero. Quindi **possiamo contare le macchine di Turing**, chiamandole $M_1, M_2, M_3, \ldots$

2.5.2 Vettore caratteristico e argomento diagonale

Definition 2.5.1: Vettore caratteristico

Il **vettore caratteristico** del linguaggio $\mathcal{L}$ è una stringa binaria infinita $\chi_{\mathcal{L}}$ tale che la posizione i-esima vale 1 se $w_i \in \mathcal{L}$, e 0 altrimenti.

$$\chi_{\mathcal{L}} = \{0\ 1\ 1\ 0\ 0\ 1\ \cdots\}$$

Costruiamo ora una tabella infinita: sulle righe le MdT $M_1, M_2, \ldots$, sulle colonne le stringhe $w_1, w_2, \ldots$, e nella cella (i, j) scriviamo 1 se M_i accetta w_j, 0 altrimenti. Ogni riga è quindi il vettore caratteristico del linguaggio accettato da quella macchina.

	w_1	w_2	w_3	w_4	$\cdots$
M_1	**0**	1	0	0	$\cdots$
M_2	1	**1**	0	1	$\cdots$
M_3	0	0	**0**	1	$\cdots$
M_4	1	0	0	**1**	$\cdots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$

Concentriamoci sulla **diagonale**: prendiamo il valore (i, i) per ogni i, ottenendo la stringa $D = \{0\ 1\ 0\ 1\ \cdots\}$. Definiamo $\bar{D}$ come il suo complemento bit a bit, $\bar{D} = \{1\ 0\ 1\ 0\ \cdots\}$.
$\bar{D}$ è interpretabile come vettore caratteristico di un linguaggio. Chiamiamo $\mathcal{L}_d$ il linguaggio il cui vettore caratteristico è $\bar{D}$:

$$\mathcal{L}_d = \{w_i \mid M_i \not\models w_i\}$$

ovvero $\mathcal{L}_d$ contiene la i-esima stringa se e solo se la i-esima macchina *non* la accetta.

Proposition 2.5.1

$\mathcal{L}_d \notin RE$

Dimostrazione: Supponiamo per assurdo che esista una MdT M_k tale che $\mathcal{L}(M_k) = \mathcal{L}_d$. Chiediamoci se M_k accetta w_k:

- Se $M_k \models w_k$: allora $w_k \in \mathcal{L}(M_k) = \mathcal{L}_d$, ma per definizione $w_k \in \mathcal{L}_d \iff M_k \not\models w_k$. Contraddizione.
- Se $M_k \not\models w_k$: allora $w_k \in \mathcal{L}_d$, quindi M_k dovrebbe accettare w_k. Contraddizione.

In entrambi i casi arriviamo a una contraddizione, quindi $\nexists$ una MdT che accetti $\mathcal{L}_d$, ovvero $\mathcal{L}_d \notin RE$

Note:

Perché esistono linguaggi indecidibili? Le MdT sono enumerabili, quindi sono tante quanti $\mathbb{N}$. I linguaggi invece sono tutti i possibili sottoinsiemi delle stringhe, quindi la loro cardinalità è quella di $\mathbb{R}$. Esistono semplicemente **più linguaggi che algoritmi**.

2.5.3 Proprietà sui linguaggi

Definition 2.5.2: Complemento di un linguaggio

Sia $\mathcal{L} \subseteq \Sigma^*$, il **complemento** di $\mathcal{L}$ è:

$$\overline{\mathcal{L}} = \Sigma^* \setminus \mathcal{L}$$

Theorem 2.5.1

Se $\mathcal{L} \in R \implies \overline{\mathcal{L}} \in R$

Dimostrazione: Siccome $\mathcal{L} \in R$, esiste $M_{\mathcal{L}}$ che decide $\mathcal{L}$ (risponde sì o no in tempo finito). Costruiamo $\overline{M}$ che si comporta esattamente come $M_{\mathcal{L}}$ ma inverte la risposta:

$$w \longrightarrow M_{\mathcal{L}} \xrightarrow{\text{sì}\to\text{no, no}\to\text{sì}} \overline{M}$$

Poiché $M_{\mathcal{L}}$ termina sempre, anche $\overline{M}$ termina sempre $\implies \overline{\mathcal{L}} \in R$.

Theorem 2.5.2

Se $\mathcal{L} \in RE \wedge \overline{\mathcal{L}} \in RE \implies \mathcal{L} \in R$

Dimostrazione: Siano $M_{\mathcal{L}}$ e $M_{\overline{\mathcal{L}}}$ le macchine che accettano rispettivamente $\mathcal{L}$ e $\overline{\mathcal{L}}$. Costruiamo M' che esegue $M_{\mathcal{L}}$ e $M_{\overline{\mathcal{L}}}$ **in parallelo** su un input w:

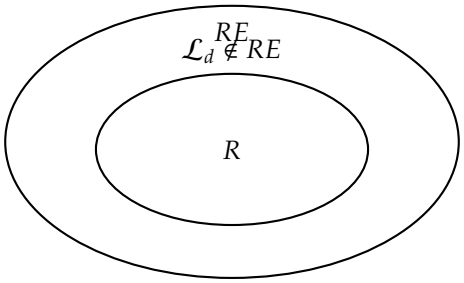
- Se risponde $M_{\mathcal{L}}$: $w \in \mathcal{L}$, accetta.
- Se risponde $M_{\overline{\mathcal{L}}}$: $w \notin \mathcal{L}$, rifiuta.

Poiché ogni stringa sta o in $\mathcal{L}$ o in $\overline{\mathcal{L}}$, una delle due macchine risponderà sempre in tempo finito $\implies M'$ è un decisore $\implies \mathcal{L} \in R$.

Note:

Questo risultato implica che RE **non è chiuso per complemento**: se lo fosse, ogni linguaggio in RE sarebbe automaticamente in R, e sappiamo che ciò non è vero.

Le classi R e RE formano quindi questa gerarchia stretta:



2.6 Linguaggio Universale e Problema della Fermata

Definition 2.6.1: Linguaggio Universale

Il linguaggio universale è l'insieme delle coppie macchina-stringa tali che la macchina M accetta w:

$$\mathcal{L}_u = \{(M, w) \mid M \models w\}$$

$\mathcal{L}_u \in RE$? Sì, perché esiste M_u che lo accetta: data la coppia (M, w), simula M su w e risponde sì se M accetta. Se M non accetta, M_u non termina, quindi nessuna garanzia sul "no".
$\mathcal{L}_u \in R$? No.

Theorem 2.6.1

$\mathcal{L}_u \notin R$

Dimostrazione: Supponiamo per assurdo che $\mathcal{L}_u \in R$, ovvero esiste $\bar{M}_u$ che *decide* $\mathcal{L}_u$. Costruiamo la macchina M':

1. Riceve w_i in input
2. Copia w_i per costruire la coppia (M_i, w_i)

3. Esegue $\bar{M}_u$ su (M_i, w_i) e **inverte** la risposta

Analizziamo $\mathcal{L}(M')$:

- M' risponde **sì** su $w_i \iff \bar{M}_u$ risponde no $\iff M_i \not\models w_i$
- M' risponde **no** su $w_i \iff \bar{M}_u$ risponde sì $\iff M_i \models w_i$

Quindi $\mathcal{L}(M') = \{w_i \mid M_i \not\models w_i\} = \mathcal{L}_d$. Ma M' termina sempre (poiché $\bar{M}_u$ termina sempre per ipotesi), quindi M' sarebbe un decisore per $\mathcal{L}_d$, il che implicherebbe $\mathcal{L}_d \in R \subseteq RE$. Contraddizione con $\mathcal{L}_d \notin RE$.

Note:

Vale anche che $\overline{\mathcal{L}_u} \notin RE$. Se così non fosse, avremmo $\mathcal{L}_u \in RE$ e $\overline{\mathcal{L}_u} \in RE$ contemporaneamente, il che implicherebbe $\mathcal{L}_u \in R$ per il teorema di chiusura — assurdo.

2.6.1 Il complemento di $\mathcal{L}_d$

Consideriamo il complemento di $\mathcal{L}_d$:

$$\bar{\mathcal{L}}_d = \{w_i \mid M_i \models w_i\}$$

Theorem 2.6.2

$\bar{\mathcal{L}}_d \in RE$

Dimostrazione: Costruiamo la macchina M: dato M_i in input, costruisce la coppia (M_i, w_i) e la passa a M_u. Se M_u accetta, risponde sì. Quindi:

- $M_i \in \bar{\mathcal{L}}_d \Rightarrow M$ risponde sì
- $M_i \notin \bar{\mathcal{L}}_d \Rightarrow M$ non risponde sì

Quindi $\bar{\mathcal{L}}_d \in RE$.

Note:

$\bar{\mathcal{L}}_d \notin R$: se lo fosse, anche $\mathcal{L}_d \in R$ per chiusura di R per complemento, da cui $\mathcal{L}_d \in RE$. Contraddizione.

2.6.2 Il Problema della Fermata

Definition 2.6.2: Problema della Fermata

$$HALT = \{(M, w) \mid M \text{ si arresta su } w\}$$

La differenza con $\mathcal{L}_u$ è sottile: in $\mathcal{L}_u$ la macchina deve fermarsi *e accettare*; in $HALT$ basta che si fermi, indipendentemente dal risultato.

Theorem 2.6.3

$HALT \in RE$

Dimostrazione: La macchina M_h accetta $HALT$: simula M su w tramite M_u, risponde sì se M si arresta. Se M non si arresta, M_h non termina. Quindi:

- $(M, w) \in HALT \Rightarrow M_h$ risponde sì
- $(M, w) \notin HALT \Rightarrow M_h$ non risponde sì

Theorem 2.6.4

$HALT \notin R$

Dimostrazione: Supponiamo per assurdo che $HALT \in R$, ovvero esiste M_h^* che decide $HALT$. Costruiamo M' su input (M, w):

1. Esegue M_h^* su (M, w)
2. Se M_h^* risponde **no**: M non si ferma su w, quindi M' risponde no
3. Se M_h^* risponde **sì**: M si ferma su w, quindi M' simula M su w tramite M_u e ne usa la risposta

In questo modo M' decide $\mathcal{L}_u$, poiché M_h^* garantisce l'arresto anche sul "no" (loop $\rightarrow$ no immediato). Quindi $\mathcal{L}_u \in R$. Contraddizione.

2.6.3 La variante $HALT_\varepsilon$

Definition 2.6.3: $HALT_\varepsilon$

$$HALT_\varepsilon = \{M \mid M \text{ si arresta su } \varepsilon\}$$

Insieme delle MdT che si arrestano quando ricevono la stringa vuota in input.

Theorem 2.6.5

$HALT_\varepsilon \in RE$

Dimostrazione: La macchina $M_{h\varepsilon}$ prende in input M, simula M su ε tramite M_u, e risponde sì se M si arresta. Quindi:

- $M \in HALT_\varepsilon \Rightarrow M_{h\varepsilon}$ risponde sì
- $M \notin HALT_\varepsilon \Rightarrow M_{h\varepsilon}$ non risponde sì

Theorem 2.6.6

$HALT_\varepsilon \notin R$

Dimostrazione: Supponiamo per assurdo che esista $M_{h\varepsilon}^*$ che decide $HALT_\varepsilon$. Costruiamo $\tilde{M}_w$: una macchina che ignora il proprio input, scrive w sul nastro e simula M su w. Costruiamo quindi M' su input (M, w):

1. Costruisce $\tilde{M}_w$ a partire da M e w
2. Esegue $M_{h\varepsilon}^*$ su $\tilde{M}_w$

Poiché $\tilde{M}_w$ su ε si comporta esattamente come M su w, abbiamo:

- M' risponde sì $\iff$ $M_{h\varepsilon}^*$ dice sì su $\tilde{M}_w$ $\iff$ $\tilde{M}_w$ si arresta su ε $\iff$ M si arresta su w
- M' risponde no altrimenti

Quindi M' decide $HALT \Rightarrow HALT \in R$. Contraddizione.

Chapter 3

Riduzioni

Le dimostrazioni dei teoremi del capitolo precedente avevano tutte un approccio simile: supponiamo per assurdo che un certo linguaggio sia decidibile, costruiamo una macchina ausiliaria sfruttando il decisore assunto esistere, e arriviamo a una contraddizione. Queste erano istanziazioni di un principio molto più generale: la **riduzione tra linguaggi**.

Definition 3.0.1: Riduzione

Siano A, B due linguaggi. La funzione f è una **riduzione** da A a B se per ogni stringa w:

$$w \in A \iff f(w) \in B$$

e inoltre f è calcolabile. Quando esiste una riduzione da A a B scriviamo $A \leqslant B$.

Note:

Intuitivamente, $A \leqslant B$ significa che A non è più difficile di B: saper risolvere B implica saper risolvere A, applicando prima f.

Per definire formalmente la calcolabilità di f è necessario introdurre un nuovo tipo di macchina.

Definition 3.0.2: Trasduttore

Un **trasduttore** è una Macchina di Turing che calcola funzioni, caratterizzata da tre nastri:

1. **Input** — sola lettura
2. **Worktape** — lettura e scrittura
3. **Output** — sola scrittura

Un trasduttore M *calcola* la funzione f se, per ogni stringa w, quando M esegue su w e si arresta, lascia $f(w)$ scritto sul nastro di output.

Note:

Distinguiamo trasduttori (calcolatori di funzioni) dalle MdT classiche (decisori). Una **funzione calcolabile** è una funzione per cui esiste un trasduttore che la calcola e che si arresta sempre in tempo finito su ogni input.

3.1 Teorema sulle riduzioni

Theorem 3.1.1

Siano A, B due linguaggi tali che $A \leqslant B$. Allora:

1. Se $A \notin R \Rightarrow B \notin R$

2. Se $A \notin RE \Rightarrow B \notin RE$

Dimostrazione: **Caso 1.** Supponiamo per assurdo che $A \notin R$ ma $B \in R$. Allora esiste M_B che decide B. Costruiamo M_A su input w:

1. Calcola $f(w)$ tramite il trasduttore per f
2. Esegue M_B su $f(w)$
3. Risponde come M_B

Il trasduttore termina sempre per definizione di funzione calcolabile, e M_B termina sempre perché decide B. La risposta è corretta poiché $w \in A \iff f(w) \in B$. Quindi M_A decide $A \Rightarrow A \in R$. Contraddizione.
Caso 2. Supponiamo per assurdo che $A \notin RE$ ma $B \in RE$. Allora esiste M_B che accetta B. Costruiamo M_A con la stessa pipeline di prima: calcola $f(w)$ e poi esegue M_B su $f(w)$. Allora:

- $w \in A \Rightarrow f(w) \in B \Rightarrow M_B$ accetta $\Rightarrow M_A$ accetta
- $w \notin A \Rightarrow f(w) \notin B \Rightarrow M_B$ non accetta $\Rightarrow M_A$ non accetta

Quindi M_A accetta $A \Rightarrow A \in RE$. Contraddizione.

Corollary 3.1.1

Siano A, B due linguaggi tali che $A \leqslant B$. Allora:

$$B \in R \Rightarrow A \in R \qquad B \in RE \Rightarrow A \in RE$$

Dimostrazione: Segue direttamente per contronominale dal teorema precedente: $(A \notin R \Rightarrow B \notin R) \equiv (B \in R \Rightarrow A \in R)$.

Note:

Nella pratica usiamo sempre il corollario in questa direzione: vogliamo dimostrare che B è difficile. Partiamo da un A noto essere indecidibile, mostriamo $A \leqslant B$, e concludiamo che B non può essere decidibile — altrimenti lo sarebbe anche A.

3.2 Esempio: $\mathcal{L}_{ne}$

Definition 3.2.1: $\mathcal{L}_{ne}$

$$\mathcal{L}_{ne} = \{M_i \mid \mathcal{L}(M_i) \neq \emptyset\}$$

L'insieme delle MdT che accettano almeno una stringa.

Theorem 3.2.1

$\mathcal{L}_{ne} \in RE$

Dimostrazione: Costruiamo la macchina M_{ne} che accetta $\mathcal{L}_{ne}$. Data M_i in input, M_{ne} non sa a priori quale stringa M_i potrebbe accettare, quindi la **indovina** non deterministicamente (design pattern *guess & check*):

1. Indovina non deterministicamente una stringa w
2. Simula M_i su w tramite M_u
3. Se M_i accetta w, risponde sì

Se $\mathcal{L}(M_i) \neq \emptyset$, esiste almeno una stringa w^* accettata da M_i: un ramo non deterministico la indovinerà e risponderà sì in tempo finito. Non diamo garanzie sul no, ma per RE non è richiesto.

Theorem 3.2.2
$\mathcal{L}_{ne} \notin R$

Dimostrazione: Mostriamo la riduzione $\mathcal{L}_u \leqslant \mathcal{L}_{ne}$. Dobbiamo costruire una funzione f calcolabile tale che:

$$(M, w) \in \mathcal{L}_u \iff f(M, w) \in \mathcal{L}_{ne}$$

La funzione f, data la coppia (M, w), costruisce una nuova macchina M' definita come segue: M' **ignora il proprio input** x, scrive w sul nastro, e simula M su w.
Verifichiamo la correttezza di f in entrambe le direzioni:
($\Rightarrow$) Supponiamo $(M, w) \in \mathcal{L}_u$, ovvero $M \models w$. Allora M', su qualsiasi input x, ignora x, simula M su w, e accetta — poiché M accetta w. Quindi $\mathcal{L}(M') = \Sigma^* \neq \emptyset$, da cui $M' \in \mathcal{L}_{ne}$.
($\Leftarrow$) Supponiamo $(M, w) \notin \mathcal{L}_u$, ovvero $M \not\models w$. Allora M', su qualsiasi input x, simula M su w e non accetta mai. Quindi $\mathcal{L}(M') = \emptyset$, da cui $M' \notin \mathcal{L}_{ne}$.
La riduzione è corretta. Poiché $\mathcal{L}_u \notin R$ e $\mathcal{L}_u \leqslant \mathcal{L}_{ne}$, per il teorema sulle riduzioni segue $\mathcal{L}_{ne} \notin R$.

3.3 Corrispondenza di Post

Andiamo a definire un problema che inizialmente sembra semplice, ma che usando le riduzioni puo' essere dimostrato che non appartiene a R: il **problema di corrispondenza di Post** (PCP).

3.3.1 Definizione

E' un problema di decisione:

- **Input**: lista di coppie di stringhe $w \in \Sigma^*$ R, S tale che $|R| = |S|$

$R(r_i)$	$S(s_i)$
1	1 1 1
1 0 1 1 1	1 0
1 0	0

- **Output**: se esiste una sequenza di indici $i_1, ..., i_u$ (con $u > 0$ per non essere banale) tale che

$$r_{i_1} \cdot ... \cdot r_{i_u} = s_{i_1} \cdot ... \cdot s_{i_u}$$

  allora l'input e' accettato.

Example 3.3.1
Per la costruzione di sopra, l'input R, S e' accettato, dato che la sequenza

$$2, 1, 1, 3$$

genera le stringhe

$$r \longrightarrow 101111110$$

e

$$s \longrightarrow 101111110$$

che sono uguali.

3.3.2 Appartenenza a RE

Per dimostrare che PCP $\in RE$, ci basta costruire una macchina (anche non deterministica) che riconosca il linguaggio associato al problema.

La soluzione e' abbastanza semplice: basta costruire una macchina non-deterministica che "indovina" (guess) la sequenza e che ne controlli (check) la validita'.

3.3.3 Non-appartenenza a R

Dobbiamo ora vedere se PCP appartiene o meno a R, che coincide col trovare una macchina in grado di decidere il linguaggio del problema.

Intuitivamente si puo' vedere come, dato che le sequenze possibili sono infinite, il problema possa divergere. Ma vediamo come dimostrarlo:

- Riduciamo il linugaggio $\mathcal{L}_u$ a un linguaggio intermedio $MPCP$. Dato che il linguaggio universale e' indecidibile, lo e' anche $MPCP$.
- Facciamo una seconda riduzione da $MPCP$ a PCP, e per lo stesso motivo questo significa che $PCP \notin R$.

Quindi

$$\mathcal{L}_u \leqslant MPCP \leqslant PCP$$

Il linguaggio $MPCP$ e' molto simile a PCP, ma le sequenze accettate devono iniziare tutte con il primo indice.

Riduzione a MPCP

Guardiamo la prima riduzione $\mathcal{L}_u \leqslant MPCP$ e facciamoci le solite domande

1. Qual'e' il problema di partenza? E quello di arrivo?
2. Quali sono gli input dei due problemi?
3. Quali sono le istanze si dei due problemi? E le istanze no?
4. Come facciamo a trasformare istanze si del problema di partenza in istanze si del problema di arrivo e stessa cosa per le istanze no?

Sappiamo che l'input al problema di partenza sono le coppie macchina/stringa (M, w) e le sue istanze si' sono le coppie dove M accetta w, mentre le no sono quelle dove M non accetta w.

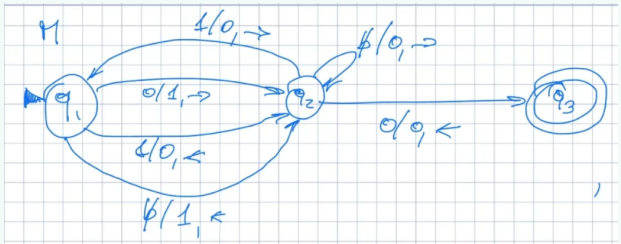
Dobbiamo trovare ora una funzione calcolabile f che prende gli input di $\mathcal{L}_u$ e li trasforma in input di $MPCP$ in modo che: se (M, w) e' un'istanza si, allora $f((M, w))$ e' un'istanza si di $MPCP$; e se e' un'istanza no, allora $f((M, w))$ e' un'istanza no di $MPCP$.

Note:

ATTENZIONE: puo' sembrare possibile definire f come la funzione che "simula" w su M e se accetta allora crea un'istanza si banale per MPCP, altrimenti ne crea una no banale. Ma questo non e' possibile dato che $\mathcal{L}_u \notin R$ quindi f puo' divergere, rendendola non-calcolabile.

Ricordiamoci che una computazione di una MdT puo' essere scritta come una sequenza di configurazioni.

Example 3.3.2



Dato in input $w = 01$, otteniamo la seguente computazione

$$q_1 01 \vdash 1q_2 1 \vdash 10q_1 \vdash 1q_2 01 \vdash q_3 101$$

dove l'ultima e' una configurazione accettante.

L'idea e' quella di guardare la funzione di transizione della macchina M e la stringa in input w per poi impostare le stringhe in R e in S in modo tale che se esiste una computazione di M per w, allora esiste una sequenza per cui $r = s$, che e' la stringa che rappresenta la computazione. Al contrario, se non esiste una computazione allora non esistera' nessuna sequenza per cui $r = s$.

Proviamo a guardare un esempio di come possiamo costruire una tale funzione sulla macchina 3.3.3:

R	S
$\#$	$\#q_1 01\#$
$q_1 0$	$1q_2$
1	1
$\#$	$\#$
$q_2 1$	$0q_1$
$0q_1\#$	$q_2 01$
$1q_2 0$	$q_3 10$
$q_3 1$	q_3
$q_3 0$	q_3
$q_3\#\#$	$\#$

Per come abbiamo costruito R e S, vediamo che c'e' una soluzione al problema MPCP che e' la seguente:

$$r = s = \#q_1 01\#1q_2 1\#10q_1\#1q_2 01\#q_3 101\#q_3 01\#q_3 1\#q_3\#\#$$

che e' proprio la computazione della stringa w con delle parti finali aggiuntive.

Abbiamo infatti costruito R in modo da essere un "passo" piu' indietro rispetto a S nella computazione, ovvero se su r_i leggo una configurazione allora su s_i avro' il suo legal successor. Quando s raggiunge lo stato accettante (quindi w e' accettata), r fa un "passo" in piu' per pareggiare le due stringhe.

In questo modo, se esiste una computazione per $w = 01$ le due stringhe possono ricongiungersi; ma se s non arriva mai ad una configurazione accettante, allora r sara' sempre un passo indietro.

Formalizziamo questa idea definendo una costruzione della coppia (R, S) a partire da (M, w), distinguendo 5 parti:

	R	S	
	$\#$	$\#q_0 w_1 ... w_n$	
$\forall x \in \Gamma$	x	x	
	$\#$	$\#$	
$\forall q \in Q \setminus F,$	qx	yp	$\delta(q, x) = (p, y, \rightarrow)$
$\forall x, y, z \in \Gamma$	zqx	pzy	$\delta(q, x) = (p, y, \leftarrow)$
	$q\#$	$yp\#$	$\delta(q, \not{b}) = (p, y, \rightarrow)$
	$zq\#$	$pzy\#$	$\delta(q, \not{b}) = (p, y, \leftarrow)$
$\forall q \in F,$	xqy	q	
$\forall x, y, z \in \Gamma$	xq	q	
	qy	q	
$\forall q \in F$	$q\#\#$	$\#$	

Controlliamo che f sia effettivamente la trasformazione che stavamo cercando:

- E' una funzione calcolabile?
  Si', dato che che non ci serve sapere se M accetta w, ma costruisce solo un numero finito di coppie di stringhe guardando la funzione di transizione di M.
- Se $(M, w) \in \mathcal{L}_u$, e' vero che $f(M, w)$ e' un'istanza si' di $MPCP$?
  Si', dato che per ogni w accettata da M esiste una sequenza che simula l'esecuzione di M su w.
- Se $(M, w) \notin \mathcal{L}_u$, e' vero che $f(M, w)$ e' un'istanza no di $MPCP$?
  Si', dato che se $f(M, w) = (R, S)$ e' un'istanza si di $MPCP$, allora esiste una sequenza che inizia con la configurazione iniziale di M su w e finisce in una configurazione accettante rispettando la funzione di transizione di M, quindi M accetta w quindi $(M, w) \in \mathcal{L}_u$. Dato che abbiamo dimostrato il contra-positivo, abbiamo dimostrato la premessa iniziale.

Riduzione a PCP

Ci basta ora trovare la seconda riduzione $MPCP \leqslant PCP$.
Dopo aver risposto alle solite domande, non rimane che costruire la funzione f:

R	S
1	111
10111	10
10	0

$\longrightarrow$

T	U
1	*1*1*1
1*	*1*1*1
1*0*1*1*1*	*1*0
1*0*	*0
$	*$

Stiamo aggiungendo un carattere speciale a sinistra di ogni carattere delle stringhe in R e a destra di ogni carattere delle stringhe in S. In questo modo e' impossibile chesia un'istanza si di PCP dato che t iniziera' sempre con il carattere speciale e u no.

Aggiungiamo quindi una coppia $t_0 = *t_1, u_0 = u_1$ in modo che, se esiste una sequenza che risolva il problema, allora il primo indice della sequenza i_1 e' per forza 0.

Dobbiamo inoltre aggiungere un carattere speciale alla fine di u, quindi aggiungiamo una coppia $t_{k+1} = \$, u_{k+1} = *\$$ in modo che, se esiste una sequenza che risolva il problema, allora l'ultimo indice di tale sequenza e' per forza $k+1$.

Controlliamo che f sia effettivamente la trasformazione che stavamo cercando:

- E' una funzione calcolabile?
  Si', non fa nulla di crazy.
- Se (R, S) e' un'istanza si' di $MPCP$, e' vero che $f(R, S) = (T, U)$ e' un'istanza si' di PCP?
  Si', dato che sia $r_1 r_{i_2} ... r_{i_n} = s_1 s_{i_2} ... s_{i_n}$ una sequenza di (R, S), allora la sequenza $t_0 t_{i_2} ... t_{i_n} t_{k+1} = u_0 u_{i_2} ... u_{i_n} u_{k+1}$ e' un'istanza si' di PCP.
- Se (R, S) e' un'istanza no di $MPCP$, e' vero che $f(R, S)$ e' un'istanza no di $MPCP$?
  Si', dato che se $f(R, S) = (T, U)$ e' un'istanza si di PCP, allora per le proprieta' di f indicate prima, t e u devono iniziare con t_0/u_0 e finire con t_{k+1}/u_{k+1}:

  $$t_0 t_{i_2} ... t_{i_n} t_{k+1} = u_0 u_{i_2} ... u_{i_n} u_{k+1}$$

  Tralasciando l'ultimo indice e considerando $i_1 = 1$, vediamo come

  $$r_1 r_{i_2} ... r_{i_n} = s_1 s_{i_2} ... s_{i_n}$$

  e' una soluzione al problema $MPCP$, quindi (R, S) e' un'istanza si. Dato che abbiamo dimostrato il contrapositivo, abbiamo dimostrato la premessa iniziale.

3.4 Il Problema del Tassellamento

3.4.1 Definizione informale

Si consideri un piano quadrettato infinito (il primo quadrante di $\mathbb{N} \times \mathbb{N}$). Abbiamo un numero finito di *tipi di piastrelle*, ciascuna con i quattro lati colorati. Una piastrella speciale S (o d_0) deve essere posizionata nell'origine $(0, 0)$.
Le piastrelle possono essere affiancate *solo "a domino"*: i bordi a contatto devono avere lo stesso colore. **Non è consentita la rotazione.**

$\longrightarrow$ possono essere affiancate solo "a domino"

Domanda: È possibile ricoprire l'intero primo quadrante affiancando le piastrelle (senza ruotarle)?

3.4.2 Formalizzazione

Definition 3.4.1: Sistema di tassellamento

Un *sistema di tassellamento* è una quadrupla

$$T = (D,\, d_0,\, H,\, V)$$

dove:

- D è l'insieme finito dei *tipi di piastrelle*;
- $d_0 \in D$ è la *piastrella speciale* (l'unica che può stare in posizione $(0,0)$);
- $H \subseteq D \times D$ sono le *regole di affiancamento orizzontale*: $(a,b) \in H$ significa che a può stare immediatamente a sinistra di b;
- $V \subseteq D \times D$ sono le *regole di affiancamento verticale*: $(a,b) \in V$ significa che a può stare immediatamente sotto b.

Definition 3.4.2: Tassellamento valido

Una funzione $f : \mathbb{N} \times \mathbb{N} \to D$ è un *tassellamento valido* per T se:

$$f(0,0) = d_0, \tag{3.1}$$

$$\big(f(m,n),\ f(m+1,n)\big) \in H \quad \forall m,n \in \mathbb{N}, \tag{3.2}$$

$$\big(f(m,n),\ f(m,n+1)\big) \in V \quad \forall m,n \in \mathbb{N}. \tag{3.3}$$

Il **Problema del Tassellamento** (Tiling) è:

$$\text{Tiling} = \{\, T = (D, d_0, H, V) \mid \exists f : \mathbb{N} \times \mathbb{N} \to D \text{ tassellamento valido per } T \,\}.$$

3.4.3 Indecidibilita' del Tassellamento

Vogliamo dimostrare che Tiling è indecidibile. Per farlo mostriamo che

$$\overline{\text{HALT}_\epsilon} \leqslant \text{Tiling},$$

dove $\text{HALT}_\epsilon = \{\, M \mid M \text{ si ferma su } \varepsilon \,\}$ e $\overline{\text{HALT}_\epsilon ps}$ è il suo complementare (le macchine che *non* si fermano su ε).
Poiché $\overline{\text{HALT}_\epsilon ps} \notin \text{RE}$, dalla riduzione segue che Tiling $\notin$ RE, in particolare è indecidibile.
Definiamo una funzione calcolabile

$$g : \text{Macchine di Turing} \longrightarrow \text{Sistemi di tassellamento}, \quad M \longmapsto T = g(M),$$

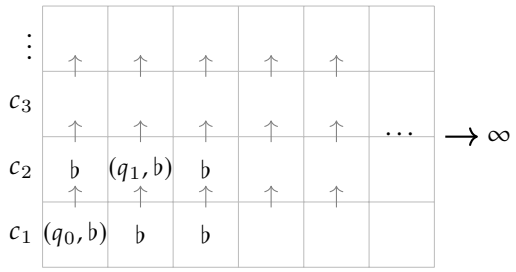
tale che:

- Se M **non si ferma** su ε (cioè $M \notin \text{HALT}_\epsilon$), allora $\exists f$ per T, ossia $T \in$ Tiling.
- Se M **si ferma** su ε (cioè $M \in \text{HALT}_\epsilon ps$), allora $\nexists f$ per T, ossia $T \notin$ Tiling.

Se M diverge su ε, la computazione è infinita:

$$c_1 \vdash c_2 \vdash c_3 \vdash \cdots$$

Ogni configurazione c_i viene "tradotta" in una riga del piano.



Intuizione: se la sequenza $c_1, c_2, \ldots$ è infinita, allora è possibile piastrellare tutto il piano.

Colori (lati delle piastrelle). I colori codificano coppie (stato, simbolo) oppure semplici simboli del nastro:

$$\underbrace{q_0\mathfrak{b}, \quad \mathfrak{b}, \quad q_1\mathfrak{b}, \quad \ldots,}_{\text{coppie (stato, simbolo)}} \qquad \underbrace{\vec{q}_1, \quad \vec{q}_0, \quad \ldots}_{\text{stati con direzione}}$$

Tipi di piastrelle. Per codificare la computazione di M costruiamo D, H e V come segue. Ogni piastrella è un quadratino con quattro lati colorati (basso, alto, sinistra, destra). Usiamo la notazione diagonale:

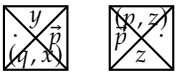
Costruiamo tre famiglie di piastrelle:

1) **Piastrelle di propagazione (per ogni $x \in \Gamma$).**
   Per ogni simbolo x del nastro, una piastrella che porta il simbolo al livello superiore senza modificarlo:

$\forall x \in \Gamma$

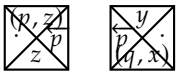
   Questi permettono di portare i simboli al livello sopra.

2) **Piastrelle di transizione $\rightarrow$ (mossa a destra).**
   Per ogni transizione $\delta(q, x) = (p, y, \rightarrow)$ e per ogni $z \in \Gamma$, si generano due piastrelle adiacenti:



$\forall z \in \Gamma$

3) **Piastrelle di transizione $\leftarrow$ (mossa a sinistra).**
   Per ogni transizione $\delta(q, x) = (p, y, \leftarrow)$ e per ogni $z \in \Gamma$:



$\forall z \in \Gamma$

4) **Piastrella iniziale d_0.**
   La piastrella speciale che forza la prima riga a contenere $c_1 = (q_0, \mathfrak{b})$:

$= d_0$

Correttezza della riduzione

> **Lenma 3.4.1**
> La funzione g è calcolabile.

Dimostrazione: D, H, V si calcolano in modo effettivo a partire dalla descrizione di M: basta enumerare le transizioni e generare le piastrelle corrispondenti.

Theorem 3.4.1

$M \notin \text{HALT}_e ps \iff g(M) \in \text{Tiling}$.

Dimostrazione: ($\Rightarrow$) Sia M una macchina che *non* si ferma su ε. Allora la computazione è infinita: $c_1 \vdash c_2 \vdash \cdots$

Per definizione di $g(M) = T$, abbiamo la possibilità di estendere le righe all'infinito: la riga n del tassellamento codifica la configurazione c_n. La funzione $f(m, n) =$ (tipo della cella m nella configurazione c_n) è un tassellamento valido per T.

($\Leftarrow$) Sia $T = g(M)$ ammette un tassellamento f. Per costruzione, la riga 0 è forzata a contenere le piastrelle speciali che codificano c_1. Ogni riga deve essere la configurazione successiva, dunque f codifica una computazione infinita $c_1 \vdash c_2 \vdash \cdots$, il che implica $M \notin \text{HALT}_e ps$.

Per contrapposizione: $M \in \text{HALT}_e ps \Rightarrow g(M) \notin \text{Tiling}$.

Corollary 3.4.1

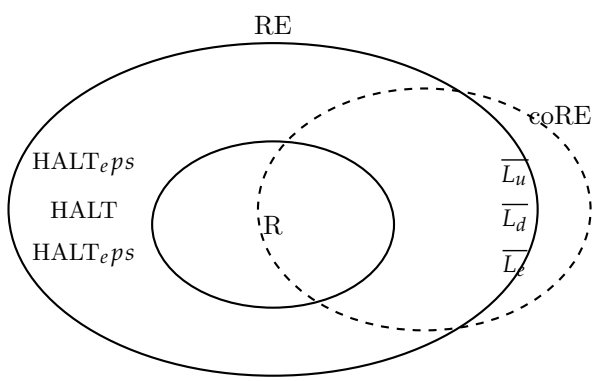
Il Problema del Tassellamento è indecidibile.

3.5 Teoria della Ricorsione: gerarchia RE/R/co-RE

3.5.1 Riepilogo delle classi

Definition 3.5.1

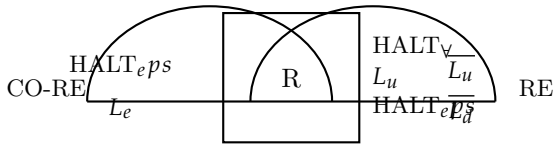
$$\begin{aligned} \text{R} &= \{\, L \mid L \text{ è decidibile} \,\} \\ \text{RE} &= \{\, L \mid L \text{ è semidecidibile (riconoscibile)} \,\} \\ \text{coRE} &= \{\, L \mid \overline{L} \in \text{RE} \,\} \end{aligned}$$



Esempi notevoli.

- $L_u = \{(M, w) \mid M \text{ si ferma su } w\}$: è un problema t.c. in RE, abbiamo una procedura per rispondere sì in tempo finito.
- $\overline{L_u}$: è un problema che deve arrivare all'infinito per dire sì, quindi $\overline{L_u} \notin \text{RE}$.

Tutti questi problemi hanno il complementare in RE, quindi possiamo dire no in tempo finito. Definiamo così due classi:



Cosa sta fuori? Non possiamo dire né sì né no.
(non è comunque il più difficile)

3.5.2 Teoremi fondamentali

Theorem 3.5.1
R $\neq$ RE

Dimostrazione: $\exists L \in$ RE con $L \notin$ R: ad esempio HALT$_e ps \in$ RE $\setminus$ R.

Theorem 3.5.2
coRE $\cap$ RE = R

Dimostrazione: **(coRE $\cap$ RE $\subseteq$ R):** Sia $L \in$ coRE $\cap$ RE. Se $L \in$ coRE, allora $\overline{L} \in$ RE. Ma se $L \in$ RE e $\overline{L} \in$ RE, allora $L \in$ R (si eseguono in parallelo i due semidecisori e si risponde quando uno dei due si ferma).
(R $\subseteq$ coRE $\cap$ RE): Sia $L \in$ R; allora esiste H che decide L. Tale H risponde sì ($\in$ RE) e risponde no ($\in$ coRE).

Theorem 3.5.3
RE $\neq$ coRE

Dimostrazione: Per il Teorema 3.5.2, coRE $\cap$ RE = R. Se fosse RE = coRE, allora RE = R, contraddicendo il Teorema 3.5.2.

3.5.3 Il problema HALT$_\forall$

Definition 3.5.2

$$\text{HALT}_\forall = \{\, M \mid M \text{ si ferma su tutte le stringhe} \,\}$$

- HALT$_\forall \notin$ RE: per rispondere sì bisognerebbe provarlo su tutte le stringhe (infinitely many).
- HALT$_\forall \notin$ coRE: per rispondere no bisognerebbe dimostrare che M non si ferma su una stringa, ma questo è indecidibile.

Dunque HALT$_\forall$ sta *fuori* sia da RE che da coRE.

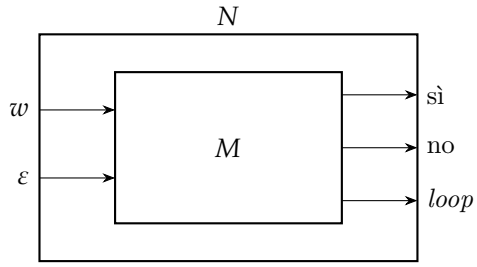
3.6 Riduzione di HALT$_\forall$ fuori da RE e coRE

Usiamo il principio: $A \leqslant B \Rightarrow (A \notin \text{RE} \Rightarrow B \notin \text{RE})$. Dimostriamo due riduzioni.

3.6.1 Riduzione 1: HALT$_e ps \leqslant$ HALT$_\forall$

Conseguenza: HALT$_e ps \in$ coRE e HALT$_e ps \leqslant$ HALT$_\forall$, quindi HALT$_\forall \notin$ coRE.

Costruzione. Data M, costruiamo $N = f(M)$ tale che:



N su input w: simula M su ε, ignorando w.

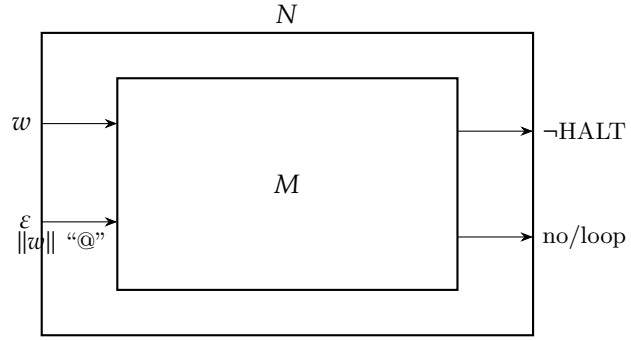
- Se M non si ferma su ε: N non si ferma mai $\Rightarrow N \notin \mathrm{HALT}_\forall$.
- Se M si ferma su ε: N si ferma su ogni $w \Rightarrow N \in \mathrm{HALT}_\forall$.

Quindi: $M \in \mathrm{HALT}_e ps \Leftrightarrow N \in \mathrm{HALT}_\forall$.

3.6.2 Riduzione 2: $\overline{\mathrm{HALT}_e ps} \leqslant \mathrm{HALT}_\forall$

Conseguenza: $\overline{\mathrm{HALT}_e ps} \notin \mathrm{RE}$ (poiché $\overline{\mathrm{HALT}_e ps} = \mathrm{coRE} \setminus \mathrm{RE}$), quindi $\mathrm{HALT}_\forall \notin \mathrm{RE}$.

Costruzione. Data M, costruiamo $N = f(M)$ tale che:



N su input w: simula M su ε per $|w|$ passi (usando la lunghezza di w come "conto alla rovescia").

- Se M **non** si ferma su ε: la simulazione non termina mai entro $|w|$ passi per nessun w, quindi N si ferma sempre $\Rightarrow N \in \mathrm{HALT}_\forall$.
- Se M **si ferma** su ε in k passi: per ogni w con $|w| \geqslant k$, la simulazione completa e N diverge su w $\Rightarrow N \notin \mathrm{HALT}_\forall$.

Quindi: $M \notin \mathrm{HALT}_e ps \Leftrightarrow N \in \mathrm{HALT}_\forall$, cioè $\overline{\mathrm{HALT}_e ps} \leqslant \mathrm{HALT}_\forall$.

Conclusione.

$$\mathrm{HALT}_\forall \notin \mathrm{RE} \quad \text{e} \quad \mathrm{HALT}_\forall \notin \mathrm{coRE}$$

3.7 Teorema di Rice

Il **Teorema di Rice** è un meta-teorema che identifica l'indecidibilità dei linguaggi che contengono stringhe che codificano macchine di Turing.

3.7.1 Proprietà di macchina

Definition 3.7.1: Proprieta' di macchina

Una **proprietà di macchina** è un insieme P di codifiche di macchine di Turing.

Ad essa si associa il linguaggio

$$L_P = \{M \mid M \in P\}$$

che descrive il problema di *decidere la proprietà P di una macchina*.
Un linguaggio L è una proprietà di macchine se contiene *solo* codifiche di macchine di Turing.

Definition 3.7.2: Proprieta' di macchina banale

Una proprietà P di macchine è **banale** se:

- $P = \emptyset$, oppure
- $P =$ tutte le macchine di Turing.

Decidere queste proprietà è ovviamente triviale.

Example 3.7.1 (Proprieta' di macchina)

$$P_1 = \{M \mid M \text{ ha 5 stati}\} \quad \text{(contiene codifiche di MdT} \Rightarrow \text{proprietà di macchina; non è banale)}$$
$$P_2 = \{\langle M \rangle \mid \mathcal{L}(M) \text{ contiene solo stringhe di lunghezza pari}\}$$
$$P_3 = \{\langle M \rangle \mid \mathcal{L}(M) = \emptyset\} = L_e$$

Si noti che P_1 riguarda la *struttura* della macchina, mentre P_2 e P_3 riguardano il *linguaggio* della macchina.

3.7.2 Proprietà semantica (di linguaggio)

Definition 3.7.3: Proprieta' semantica

Sia P una proprietà di macchine. P è una **proprietà semantica** se

$$\forall M_1, M_2.\ \mathcal{L}(M_1) = \mathcal{L}(M_2) \ \Rightarrow\ \big(M_1 \in P \Leftrightarrow M_2 \in P\big).$$

Quindi, anziché come un insieme di macchine, possiamo vedere P come un insieme di linguaggi:

$$P \subseteq \mathrm{RE}$$

dove RE è la classe dei linguaggi riconoscibili da una macchina di Turing.

Di conseguenza, P è banale se e solo se $P = \emptyset$ oppure $P = \mathrm{RE}$.

Decidere la proprietà P significa decidere il linguaggio

$$L_P = \{M \mid \mathcal{L}(M) \in P\},$$

dove le MdT vengono usate come codifiche finite di linguaggi che potrebbero essere infiniti.

3.7.3 Teorema di Rice

Theorem 3.7.1 Rice

Ogni proprietà non banale di linguaggi RE è indecidibile.

Dimostrazione: Sia $P \subseteq \mathrm{RE}$ una proprietà di linguaggio non banale. La dimostrazione si divide in due casi a seconda che $\emptyset \in P$ o meno.

Caso (a): $\emptyset \notin P$.
Poiché $P \neq \emptyset$ (altrimenti sarebbe banale), esiste $L \neq \emptyset$ con $L \in P$; sia M_L una MdT tale che $\mathcal{L}(M_L) = L$.
Mostriamo che

$$L_P = \{\langle M \rangle \mid \mathcal{L}(M) \in P\}$$

è indecidibile ($L_P \notin R$), riducendo L_u a L_P:

$$L_u \leqslant L_P, \qquad (M, w) \overset{f}{\longmapsto} N = f(M, w).$$

La macchina $N = f(M, w)$ è definita nel modo seguente: dato un input x,

1. simula M su w;
2. se M accetta w, avvia M_L su x e ne restituisce il risultato.

Il comportamento è illustrato dallo schema seguente.
TODO: immagine macchina

Correttezza della riduzione.

($\Rightarrow$) Supponiamo $(M, w) \in L_u$, ovvero $M \vdash w$. In questo caso N si comporta come M_L sulla codifica di M; quindi $\mathcal{L}(N) = \mathcal{L}(M_L) = L \in P$, da cui $N \in L_P$.

($\Leftarrow$) Supponiamo $(M, w) \notin L_u$, ovvero $M \nvdash w$. Allora il primo passo di N non termina su nessun input, quindi

$$\mathcal{L}(N) = \emptyset.$$

Poiché $\emptyset \notin P$, si ha $N \notin L_P$.

Dunque $L_u \leqslant L_P$ e L_P è indecidibile.

Caso (b): $\emptyset \in P$.
Consideriamo il complemento $\overline{P} = \mathrm{RE} \setminus P$. Poiché $\emptyset \notin \overline{P}$ e $\overline{P}$ non è banale (in quanto P non è banale), dal Caso (a) $L_{\overline{P}}$ è indecidibile.
Ora osserviamo che

$$\overline{L_{\overline{P}}} = L_P.$$

Supponiamo per assurdo che $\overline{L_{\overline{P}}} \in R$; allora anche $L_{\overline{P}} \in R$, contraddizione. Quindi $\overline{L_{\overline{P}}} = L_P$ è indecidibile.

Chapter 4

Complessità Strutturale

Finora abbiamo studiato le classi RE e R, occupandoci di stabilire se un problema fosse decidibile o meno. D'ora in poi ci concentreremo esclusivamente sui problemi $\in R$, ovvero sui problemi *decidibili*, e ci chiederemo quanto siano *difficili* tra loro.
Questa area di ricerca nasce da una questione storica e tecnologica: negli anni '30, con Turing, tutto era molto astratto. La domanda era "se mai fossi in grado di costruire questa macchina, che problemi sarebbe in grado di risolvere?". Quando nei primi anni '60 le macchine reali arrivarono, la gente si trovava lì ad aspettare le risposte e si rese conto che certi problemi richiedevano molto più tempo di altri. Nacque allora la domanda fondamentale:

"L'algoritmo che ho è lento perché non sono furbo a sufficienza, o il problema è talmente complicato che non riuscirò mai ad avere un algoritmo veloce?"

Questo fu formalizzato da **Hartmanis, Stearns e Lewis nel 1965**, introducendo i concetti di **complessità temporale** e **complessità spaziale**. Lo studio si chiama **complessità strutturale** perché si vuole occupare della struttura dei problemi:

"Com'è che la struttura di un problema influenza la complessità degli algoritmi che lo risolvono?"

L'oggetto di interesse non è la complessità di un algoritmo specifico (come nel corso di Algoritmi e Strutture Dati), bensì la complessità *intrinseca del problema*, indipendentemente dall'algoritmo scelto:

Dato un problema, stabilire la famiglia di problemi di complessità simile a cui appartiene.

4.1 Definizioni di Complessità Temporale

Ci focalizzeremo sull'insieme dei problemi decidibili. L'obiettivo è arricchire la struttura di R identificando al suo interno classi di linguaggi di difficoltà differente.

Definition 4.1.1: Computation Time

Si definisce *computation time* di una MdT M su input w il numero di passi che M esegue prima di arrestarsi su w.
Se M è non deterministica, il computation time è dato dalla lunghezza del *branch di computazione più lungo* nell'albero delle computazioni.

Definition 4.1.2: Time Function

Si definisce *time function* una funzione $t : \mathbb{N} \to \mathbb{N}$ tale che $t(n)$ è non decrescente e strettamente positiva.

Definition 4.1.3: Running Time

Sia $t(n)$ una time function. Si definisce il *running time* di una MdT M pari a $t(n)$ se per tutte le stringhe w, a parte un numero finito, il computation time di M su w non eccede $t(\|w\|)$.

Note:

L'eccezione per un numero finito di stringhe serve a non far dipendere la definizione da casi limite irrilevanti: ci interessa il comportamento asintotico, non gli outlier.

4.1.1 Notazione Asintotica

Definition 4.1.4: Big O

Si definisce *Big O*: una funzione $f(n) \in O(g(n))$ per un'altra funzione $g(n)$ se:

$$\exists\, c, n_0 \;:\; \forall n \geqslant n_0, \quad f(n) \leqslant c \cdot g(n)$$

Rappresenta un *limite asintotico superiore*.

Definition 4.1.5: Big Ω

Si definisce *Big Omega*: una funzione $f(n) \in \Omega(g(n))$ per un'altra funzione $g(n)$ se:

$$\exists\, c, n_0 \;:\; \forall n \geqslant n_0, \quad f(n) \geqslant c \cdot g(n)$$

Rappresenta un *limite asintotico inferiore*.

Definition 4.1.6: Big Θ

Si definisce *Big Theta*: una funzione $f(n) \in \Theta(g(n))$ se $f(n) \in O(g(n))$ e $f(n) \in \Omega(g(n))$ simultaneamente. Rappresenta un *limite asintotico stretto*.

4.1.2 Complessità di un Problema

Attraverso la notazione asintotica introduciamo il concetto centrale di complessità di un *problema*, distinguendola dalla complessità di un algoritmo specifico.

Definition 4.1.7: Time Complexity Upper Bound

Si definisce il *time complexity upper bound* di un problema P pari a $O(f(n))$ (dove $f(n)$ è una time function) se esiste almeno una MdT che risolve P con un running time che è $O(f(n))$.

Definition 4.1.8: Time Complexity Lower Bound

Si definisce il *time complexity lower bound* di un problema P pari a $\Omega(f(n))$ (dove $f(n)$ è una time function) se **tutti** gli algoritmi o tutte le MdT che risolvono P hanno un running time che è $\Omega(f(n))$.

Note:

Questa è la distinzione concettuale più importante del capitolo. Il lower bound dice che nessun algoritmo, presente o futuro, potrà mai fare meglio di così. L'upper bound dice che esiste almeno un algoritmo che fa così bene. Quando i due limiti coincidono, conosciamo la complessità *esatta* del problema.

$\Rightarrow$ la complessità temporale di un problema è legata alla complessità temp. degli algoritmi che lo risolvono.

4.2 Classe DTime e la Classe P

Definition 4.2.1: Classe di Complessità Temporale DTime

Sia $t(n)$ una time function. Si definisce la classe di complessità temporale deterministica:

$$DTime(t(n)) = \left\{ L \;\middle|\; \exists M \text{ deterministica che } \underline{\text{decide}}\ L \text{ in tempo } O(t(n)) \right\}$$

Definition 4.2.2: Classe P

Si definisce la classe P (Polynomial Time) come l'unione di tutte le classi $DTime$ con limite temporale polinomiale:

$$P = \bigcup_{c \geqslant 1} DTime(n^c)$$

P identifica i problemi *trattabili*: risolvibili in tempo polinomiale deterministico.

Note:

P contiene solo *problemi di decisione* (risposta sì/no). Conseguentemente, "ordinare un array" o "sommare due numeri" non sono elementi di P in senso formale, perché non sono problemi di decisione. Le loro versioni decisionali ("esiste un elemento $> k$?") invece appartengono a P.
Inoltre, P è **invariante rispetto ai modelli di calcolo deterministici ragionevoli**: simulare un modello avanzato su una MdT a nastro singolo, oppure su un computer RAM convenzionale, produce al massimo un incremento polinomiale nel tempo, assorbito dalla definizione stessa di P.

4.2.1 Esempi in P

Reachability

$$Reach = \left\{ \langle G, s, t \rangle \;\middle|\; \begin{array}{l} G \text{ è un grafo diretto,} \\ s, t \text{ sono nodi } \in G, \\ \exists \text{ un percorso da } s \text{ a } t \in G \end{array} \right\}$$

$Reach \in P$: è sufficiente una BFS/DFS a partire da s. L'algoritmo visita ogni nodo al più una volta, con costo $O(n^2)$ nella taglia del grafo.

Primes

$$Primes = \{ n \mid n \text{ è un numero primo} \}$$

L'algoritmo *naive* — dividere n per $2, 3, \ldots, n-1$ — sembra $O(n)$ divisioni, ma bisogna stare attenti a cosa intendiamo per "taglia dell'input". L'input è n rappresentato in **binario**: ha taglia $\|n\| = \log_2 n$ bit. Quindi il valore $n \approx 2^{\|n\|}$, e il numero di divisioni è $O(2^{\|n\|})$: **esponenziale nella taglia dell'input**.

$$n \approx 2^{\|n\|} \implies \text{algoritmo naive} : O\left(2^{\|n\|} \cdot \|n\|^k\right)$$

Nonostante questo, $Primes \in P$ dagli inizi del 2000: Agrawal, Kayal e Saxena (algoritmo **AKS**) hanno dimostrato l'esistenza di un algoritmo deterministico polinomiale, con complessità $O(n^6)$ nella taglia dell'input. Fu un risultato storico e sorprendente, atteso per decenni.

Example 4.2.1

Il numero $N = 1.000.000$ richiede circa $\log_2(10^6) \approx 20$ bit per essere rappresentato. L'algoritmo naive compie $\approx 10^6 = 2^{20}$ divisioni: raddoppiare la taglia dell'input (da 10 a 20 bit) fa esplodere il numero di operazioni da 10^3 a 10^6. Questa è la definizione di crescita esponenziale rispetto alla taglia.

4.3 Classe NTime e la Classe NP

I seguenti due problemi resistono ad algoritmi polinomiali deterministici e motivano la definizione di una nuova classe.

4.3.1 SAT (Soddisfacibilità Booleana)

Ci interessano formule in **Forma Normale Congiuntiva (CNF)**: una congiunzione di clausole, dove ogni clausola è una disgiunzione di letterali.

$$C = c_1 \wedge c_2 \wedge \cdots \wedge c_n, \qquad c_i = l_1 \vee l_2 \vee \cdots \vee l_m, \qquad l_j = x_k \text{ oppure } \neg x_k$$

Example 4.3.1

$$C = (x_1 \vee x_2 \vee \neg x_3) \wedge (\neg x_2 \vee x_4) \wedge (\neg x_3 \vee x_5 \vee \neg x_6)$$

Con l'assegnamento $x_1 = V,\ x_2 = F,\ x_5 = V$, la formula è soddisfatta.

$$Sat = \left\{\varphi \;\middle|\; \varphi \text{ è in CNF e soddisfacibile}\right\}$$

Dato un assegnamento in mano, verificare se soddisfa la formula costa tempo polinomiale: si scorrono tutte le clausole e si controlla che almeno un letterale per clausola sia vero.
Il problema è *trovare* quell'assegnamento: con n variabili ci sono 2^n assegnamenti possibili. Testarli tutti costa:

$$O\left(2^n \cdot \text{poly}(\|\varphi\|)\right)$$

Tempo esponenziale. Da ciò non possiamo concludere che $Sat \notin P$, semplicemente questo algoritmo non lo colloca in P.

4.3.2 Independent Set

Definition 4.3.1: Independent Set

Si definisce *independent set* di un grafo G un insieme S di nodi tale che nessuna coppia di nodi in S è collegata da un arco.

$$IS = \{\langle G, k\rangle \mid \exists \text{ un independent set di taglia } k \in G\}$$

Dato un insieme S, verificare che sia un IS richiede tempo quadratico (si controllano tutte le coppie). Trovarlo richiede generare tutti i $\binom{n}{k}$ sottoinsiemi di taglia k:

$$\binom{n}{k} \cdot O(n^2) \quad \Rightarrow \text{ esponenziale}$$

4.3.3 L'intuizione non deterministica

SAT e IS condividono un pattern:

1. La difficoltà sta nel *trovare* la soluzione (assegnamento, sottoinsieme).
2. Una volta trovata, *verificarla* è facile (polinomiale).

Una **MdT non deterministica** sfrutta esattamente questo: invece di esplorare tutto lo spazio delle soluzioni, *indovina* la soluzione giusta e poi la verifica. La macchina accetta se *esiste almeno un ramo* dell'albero di computazione che accetta.

Example 4.3.2 (IS su MdT non deterministica)

1. **Fase di guess**: per ogni nodo, la MdT ND sceglie non deterministicamente se includerlo in S o meno. Costo $O(n)$.
2. **Fase di check**: verifica che $|S| = k$ e che nessun arco colleghi due nodi in S. Costo $O(n^2)$.

Complessivamente: tempo **polinomiale non deterministico**. Similmente per SAT: la MdT indovina un assegnamento in $O(n)$ e lo verifica in $O(\|\varphi\|)$.

Definition 4.3.2: Classe di Complessità NTime

Sia $t(n)$ una time function. Si definisce la classe di complessità temporale non deterministica:

$$NTime(t(n)) = \{L \mid \exists M \text{ non deterministica che decide } L \text{ in tempo } O(t(n))\}$$

Definition 4.3.3: Classe NP

Si definisce la classe NP (Non-deterministic Polynomial Time) come:

$$NP = \bigcup_{c \geqslant 1} NTime(n^c)$$

NP è la classe dei problemi la cui soluzione può essere **verificata** in tempo polinomiale, equivalentemente dei problemi **risolti** in tempo polinomiale non deterministico.

Note:

Dire che un problema è in NP **non** significa che si riesce a risolverlo velocemente: significa che se qualcuno ci regala la soluzione (il *certificato*), possiamo verificarla in tempo polinomiale. È questa l'intuizione del pattern *Guess & Check*.

4.4 La Questione P vs NP

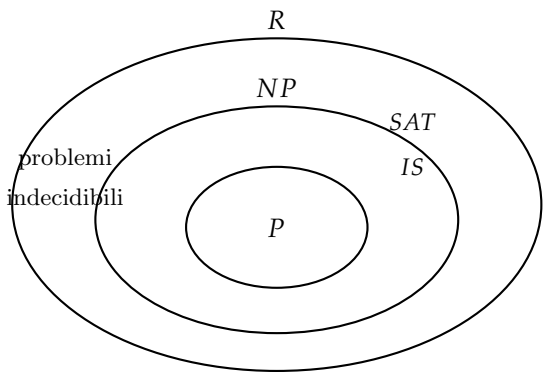
Chiaramente $P \subseteq NP$: qualsiasi problema risolvibile deterministicamente in tempo polinomiale può anche essere verificato in tempo polinomiale. La domanda aperta fondamentale è:

$$\boxed{P \stackrel{?}{=} NP}$$

Ovvero: ogni problema la cui soluzione è facilmente *verificabile* è anche facilmente *trovabile*? La risposta è quasi certamente no, ma non è mai stata dimostrata. È uno dei sette Millennium Prize Problems (Clay Mathematics Institute, 2000).

Note:

Un teorema centrale in questa direzione è il risultato di **Baker, Gill e Solovay (1975)**: esistono due oracoli A e B tali che $P^A = NP^A$ ma $P^B \neq NP^B$. Questo dimostra che qualsiasi tecnica di dimostrazione *relativizzante* (come la diagonalizzazione pura) non potrà mai risolvere la questione P vs NP, poiché qualunque risultato ottenuto con tali tecniche dovrebbe valere per entrambi gli oracoli simultaneamente.



4.5 Riduzioni Polinomiali

Definition 4.5.1: Riduzione Polinomiale

Siano A e B due linguaggi. Si dice che A si *riduce polinomialmente* a B ($A \leqslant_P B$) se $\exists f : \Sigma^* \to \Sigma^*$ tale che:

1. f e' calcolabile in tempo polinomiale deterministico
2. $\forall w \in \Sigma^*$
$$w \in A \iff f(w) \in B$$

Definition 4.5.2: NP-Hardness

Un linguaggio A si dice *NP-Hard* se $\forall L \in NP$

$$L \leqslant_P A$$

Definition 4.5.3: NP-Completeness

Un linguaggio A si dice *NP-Complete* se

1. A e' NP-Hard
2. $A \in NP$

Theorem 4.5.1 Collasso di NP

Sia L un linguaggio NP-Complete, allora

$$L \in P \iff P = NP$$

Dimostrazione:

- $\Longleftarrow$) Abbiamo che $L \in NP$ dato che e' NP-Complete, ma per ipotesi $NP = P$ quindi $L \in P$.
- $\Longrightarrow$) Dato che L e' NP-Complete sappiamo che tutti i linguaggi NP si riducono polinomialmente a L. Ma per ipotesi L e' risolvibile in tempo polinomiale deterministico, quindi anche tutti i linguaggi che si riducono polinomialmente a esso appartengono a P, quindi $NP \subseteq P$. Se un problema e' risolvibile con una macchina deterministica, allora esiste sicuramente una macchina non deterministica che lo risolve, quindi $P \subseteq NP$. Quindi $P = NP$.

Theorem 4.5.2 Transitivita' delle riduzioni polinomiali

Siano A, B e C tre linguaggi. Se $A \leqslant_P B$ e $B \leqslant_P C$, allora

$$A \leqslant_P C$$

Dimostrazione: Basta considerare la composizione delle funzioni garantite dalla riduzione. Dato che queste sono deterministiche polinomiali, lo sara' anche la composizione.

Theorem 4.5.3 Riduzioni NP-Hard

Dati due linugaggi A, B con A NP-Hard tali che $A \leqslant_P B$, allora

$$B \text{e' NP-Hard}$$

Dimostrazione: Sfruttiamo la transitivita' della riduzione polinomiale per dimostrare che se $\forall L \in NP. L \leqslant_P A$ e $A \leqslant_P B$ allora B e' NP-Hard.

Useremo questa proprieta' per dimostrare che un linugaggio e' NP-Hard, dato che trovare una riduzione per ogni linugaggio in NP e' impossibile (ce ne stanno infiniti). Ci serve pero' almeno un linugaggio che sappiamo essere NP-Hard verso il quale ci possiamo ridurre, e questo e' proprio SAT (vedremo piu' avanti come si puo' dimostrare). Per facilitarci la vita useremo un linugaggio simile, 3SAT, che dimostriamo essere NP-Hard riducendoci SAT

Definition 4.5.4: 3SAT

Come SAT, ma le formule sono in 3CNF (ogni clausola c_i ha lunghezza massima di 3 letterali)

$$\text{3STA} = \{\psi | \psi \text{ e' in 3CNF ed e' soddisfacibile}\}$$

Proposition 4.5.1 NP-Completezza di 3SAT

3SAT e' NP-Completo

Dimostrazione: Allo stesso modo di SAT, 3SAT appartiene a NP. Dobbiamo dimostrare che e' NP-Hard, che facciamo dimostrando $SAT \leqslant_P 3SAT$. Possiamo fare cio' usando una funzione abbastanza ez che a ogni passaggio riesce a diminuire di 1 la lunghezza delle clausole troppo lunghe, quindi dopo un numero finito di riduzioni riusciamo a dimostrare la riduzione iniziale (usando la prop. transitiva).

Chapter 5

Complessita Spaziale

Chapter 6

Oracoli e Classi Funzionali